PulseFit Gym and Fitness Class Booking API

Complete code listing of the back-end API (Python Flask + MongoDB)

Generated 12 October 2026

#	File	Lines
1	app.py	53
2	globals.py	33
3	decorators.py	76
4	requirements.txt	7
5	utils/errors.py	65
6	utils/validation.py	277
7	utils/serialize.py	33
8	utils/pagination.py	47
9	utils/responses.py	25
10	utils/constants.py	27
11	utils/membership.py	70
12	blueprints/auth/auth.py	138
13	blueprints/branches/branches.py	196
14	blueprints/exercises/exercises.py	180
15	blueprints/reviews/reviews.py	114
16	blueprints/members/members.py	326
17	blueprints/workouts/workouts.py	150
18	blueprints/classes/classes.py	256
19	blueprints/bookings/bookings.py	171
20	blueprints/reports/reports.py	314
21	Raw_Dataset/seed.py	508
	Total	**3066**

Raw_Dataset/seed.py builds the database from the Kaggle CSV files; it is included for completeness. Long lines are wrapped and marked with >>.

app.py

```
"""
app.py - entry point for the PulseFit Gym and Fitness Class Booking API.

Creates the Flask app, installs the MongoDB-aware JSON encoder and the JSON error
handlers, and registers one Blueprint per resource under /api/v1.0.

Run with:  python app.py   (serves on http://127.0.0.1:5000)
"""

from flask import Flask
from flask_cors import CORS

from globals import blacklist
from utils.errors import register_error_handlers
from utils.serialize import MongoJSONProvider

from blueprints.auth.auth import auth_bp
from blueprints.branches.branches import branches_bp
from blueprints.exercises.exercises import exercises_bp
from blueprints.reviews.reviews import reviews_bp
from blueprints.members.members import members_bp
from blueprints.workouts.workouts import workouts_bp
from blueprints.classes.classes import classes_bp
from blueprints.bookings.bookings import bookings_bp
from blueprints.reports.reports import reports_bp

API_PREFIX = "/api/v1.0"

BLUEPRINTS = [auth_bp, branches_bp, exercises_bp, reviews_bp, members_bp, workouts_bp,
              classes_bp, bookings_bp, reports_bp]


def ensure_indexes():
    """Indexes the API relies on that seed.py does not create (create_index is idempotent)."""
    # TTL index: MongoDB deletes each blacklisted token once its expiry time has passed
    blacklist.create_index("expires_at", expireAfterSeconds=0)


def create_app():
    app = Flask(__name__)
    app.json = MongoJSONProvider(app)     # ObjectId / datetime -> JSON
    CORS(app)                             # allow a front end on another origin to call the API
    register_error_handlers(app)
    for bp in BLUEPRINTS:
        app.register_blueprint(bp, url_prefix=API_PREFIX)
    ensure_indexes()
    return app


app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

globals.py

```
"""
globals.py - shared configuration and the MongoDB connection.

Settings come from environment variables (or the project .env file) so that no
secrets live in the code. The same MONGODB_URI variable is used by seed.py, so the
API always talks to the database that was seeded.
"""

import os

from dotenv import load_dotenv
from pymongo import MongoClient

# Load KEY=value pairs from the .env next to this file. Variables already set in the
# environment take priority, so `MONGODB_URI=... python app.py` overrides the file.
load_dotenv(os.path.join(os.path.dirname(os.path.abspath(__file__)), ".env"))

MONGO_URI = os.environ.get("MONGODB_URI", "mongodb://127.0.0.1:27017")
DB_NAME = os.environ.get("DB_NAME", "gymDB")
SECRET_KEY = os.environ.get("SECRET_KEY", "pulsefit-dev-secret-change-me")
TOKEN_EXPIRY_MINUTES = 30

# tz_aware=True makes pymongo return timezone-aware UTC datetimes, so they can be
# compared safely with datetime.now(timezone.utc) anywhere in the code.
client = MongoClient(MONGO_URI, tz_aware=True)
db = client[DB_NAME]

users = db.users
members = db.members
exercises = db.exercises
branches = db.branches
classes = db.classes
blacklist = db.blacklist
```

decorators.py

```
"""
Authentication and authorisation decorators.

    @jwt_required             -> any logged-in user (valid, unexpired, not logged-out token)
    @role_required("admin")   -> logged in AND one of the listed roles

The decoded token is stored in flask.g.user so routes can read
g.user["user_id"], g.user["role"] and g.user["member_id"].
"""

from functools import wraps

import jwt
from bson import ObjectId
from flask import g, request

from globals import SECRET_KEY, blacklist
from utils.errors import ApiError


def jwt_required(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        token = request.headers.get("x-access-token")
        if not token:
            raise ApiError(401, "Token is missing",
                           {"x-access-token": "Log in at /api/v1.0/auth/login and send the token in
 >>   this header"})
        try:
            payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        except jwt.ExpiredSignatureError:
            raise ApiError(401, "Token has expired", {"x-access-token": "Log in again to get a new t
 >>   oken"})
        except jwt.InvalidTokenError:
            raise ApiError(401, "Token is invalid")
        if blacklist.find_one({"token": token}):
            raise ApiError(401, "Token has been logged out", {"x-access-token": "Log in again to get
 >>    a new token"})
        g.user = payload
        g.token = token
        return func(*args, **kwargs)
    return wrapper


def role_required(*roles):
    """Allows the request only if the logged-in user has one of the given roles."""
    def decorator(func):
        @wraps(func)
        @jwt_required                    # role checks always need a valid token first
        def wrapper(*args, **kwargs):
            if g.user["role"] not in roles:
                raise ApiError(403, "You do not have permission to do this",
                               {"required_role": " or ".join(roles), "your_role": g.user["role"]})
            return func(*args, **kwargs)
        return wrapper
    return decorator


# ------------------------------------------------------------------ ownership helpers

def is_admin():
    return g.user["role"] == "admin"


def ensure_self_or_admin(member_id):
    """403 unless the caller is an admin or the member who owns this record."""
    if is_admin():
        return
    if g.user.get("member_id") != str(member_id):
        raise ApiError(403, "You can only access your own member record")


def current_member_id():
    """The caller's member ObjectId, or 403 for staff accounts that have no member record."""
    member_id = g.user.get("member_id")
    if not member_id:
        raise ApiError(403, "Only member accounts can do this",
                       {"your_role": g.user["role"]})
```

```
76      return ObjectId(member_id)
```

requirements.txt

```
Flask==3.1.3
flask-cors==6.0.5
pymongo==4.18.3
dnspython==2.8.0
PyJWT==2.15.1
bcrypt==5.0.0
python-dotenv==1.2.4
```

utils/errors.py

```
"""
Standard error handling.

Every error leaves the API in the same shape:
    {"error": "<short message>", "details": {...optional field errors}}

Routes raise ApiError instead of building responses, and the handlers registered
here turn it (and Flask's own 404/405/500 errors) into JSON.
"""

import logging

from flask import jsonify, make_response
from pymongo.errors import DuplicateKeyError
from werkzeug.exceptions import HTTPException


class ApiError(Exception):
    """Raised anywhere in a request to stop processing and return an error response."""

    def __init__(self, status, message, details=None):
        super().__init__(message)
        self.status = status
        self.message = message
        self.details = details


def error_response(status, message, details=None):
    body = {"error": message}
    if details:
        body["details"] = details
    return make_response(jsonify(body), status)


def register_error_handlers(app):
    @app.errorhandler(ApiError)
    def handle_api_error(e):
        return error_response(e.status, e.message, e.details)

    @app.errorhandler(404)
    def handle_not_found(e):
        return error_response(404, "Resource not found",
                              {"description": "The requested URL does not exist on this API"})

    @app.errorhandler(405)
    def handle_method_not_allowed(e):
        allowed = sorted(getattr(e, "valid_methods", None) or [])
        return error_response(405, "Method not allowed", {"allowed_methods": allowed})

    @app.errorhandler(HTTPException)
    def handle_http_exception(e):
        # Any other HTTP error Flask raises (e.g. malformed JSON -> 400)
        return error_response(e.code, e.name, {"description": e.description})

    @app.errorhandler(DuplicateKeyError)
    def handle_duplicate(e):
        # Safety net for unique indexes (username, email) if two requests race each other
        field = next(iter((e.details or {}).get("keyValue", {}) or {"value": None}))
        return error_response(409, f"A record with this {field} already exists")

    @app.errorhandler(Exception)
    def handle_unexpected(e):
        logging.exception("Unhandled error")
        return error_response(500, "Internal server error",
                              {"description": "Something went wrong on the server"})
```

utils/validation.py

```
"""
Request validation.

Each endpoint describes its input as a schema: a dict of field name -> rules.

    REVIEW_SCHEMA = {
        "stars":   {"type": "int", "required": True, "min": 1, "max": 5},
        "comment": {"type": "string", "max_len": 500},
    }

validate_body(schema) checks the JSON body and validate_query(schema) checks the
query string. Both collect every problem before failing, reject unknown fields and
raise ApiError(400) with a "details" dict of field -> message.

Supported rule keys:
    type       string | int | number | bool | email | objectid | datetime | list | object
    required   field must be present (ignored for partial updates)
    nullable   null is accepted
    min / max  numeric range (inclusive)
    min_len / max_len   string length (min_len defaults to 1, so "" is rejected)
    choices    allowed values (matched case-insensitively, stored in canonical case)
    pattern    regular expression a string must match (pattern_msg explains it)
    items      rules for each element of a list; min_items / max_items for its length
    schema     nested schema for an object
    default    query parameters only: value used when the parameter is missing
"""

import re
from datetime import datetime, timezone

from bson import ObjectId
from flask import request

from utils.errors import ApiError

EMAIL_RE = re.compile(r"^[A-Za-z0-9._%+'-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")


class FieldError(Exception):
    """One field failed validation. message is a string, or a dict for nested fields."""

    def __init__(self, message):
        super().__init__(message)
        self.message = message


# ---------------------------------------------------------------- single values

def now_utc():
    return datetime.now(timezone.utc)


def parse_datetime(value):
    """Accepts ISO 8601 dates ("2026-10-20") and date-times ("2026-10-20T18:00:00Z")."""
    if not isinstance(value, str):
        raise ValueError
    text = value.strip()
    if text.endswith(("Z", "z")):
        text = text[:-1] + "+00:00"        # Python 3.10's fromisoformat does not accept Z
    dt = datetime.fromisoformat(text)
    if dt.tzinfo is None:                  # no offset given: treat as UTC
        dt = dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(timezone.utc)


def parse_object_id(value, label="id"):
    """Turns a URL segment or field into an ObjectId, or raises 400 with a clear message."""
    if isinstance(value, str) and ObjectId.is_valid(value):
        return ObjectId(value)
    raise ApiError(400, f"Invalid {label}", {label: "Must be a 24-character hexadecimal ObjectId"})


def _check_choices(value, rules):
    choices = rules.get("choices")
    if not choices:
        return value
    for choice in choices:
        if isinstance(value, str) and isinstance(choice, str) and value.lower() == choice.lower():
```

```
            return choice
        if value == choice:
            return choice
    raise FieldError(f"Must be one of: {', '.join(map(str, choices))}")


def _check_range(value, rules):
    if "min" in rules and value < rules["min"]:
        raise FieldError(f"Must be at least {rules['min']}")
    if "max" in rules and value > rules["max"]:
        raise FieldError(f"Must be at most {rules['max']}")
    return value


def check_value(value, rules):
    """Validates one value against its rules and returns the cleaned value."""
    if value is None:
        if rules.get("nullable"):
            return None
        raise FieldError("Must not be null")

    kind = rules.get("type", "string")

    if kind in ("string", "email"):
        if not isinstance(value, str):
            raise FieldError("Must be a string")
        value = value.strip()
        if len(value) < rules.get("min_len", 1):
            raise FieldError("Must not be empty" if rules.get("min_len", 1) == 1
                             else f"Must be at least {rules['min_len']} characters")
        if "max_len" in rules and len(value) > rules["max_len"]:
            raise FieldError(f"Must be at most {rules['max_len']} characters")
        if kind == "email":
            if not EMAIL_RE.match(value):
                raise FieldError("Must be a valid email address")
            value = value.lower()
        if "pattern" in rules and not re.fullmatch(rules["pattern"], value):
            raise FieldError(rules.get("pattern_msg", "Has an invalid format"))
        return _check_choices(value, rules)

    if kind == "int":
        # bool is a subclass of int in Python, so True would otherwise pass as 1
        if isinstance(value, bool) or not isinstance(value, int):
            if isinstance(value, float) and value.is_integer():
                value = int(value)
            else:
                raise FieldError("Must be a whole number")
        return _check_choices(_check_range(value, rules), rules)

    if kind == "number":
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise FieldError("Must be a number")
        return _check_range(float(value), rules)

    if kind == "bool":
        if not isinstance(value, bool):
            raise FieldError("Must be true or false")
        return value

    if kind == "objectid":
        if not isinstance(value, str) or not ObjectId.is_valid(value):
            raise FieldError("Must be a 24-character hexadecimal ObjectId")
        return ObjectId(value)

    if kind == "datetime":
        try:
            value = parse_datetime(value)
        except (ValueError, TypeError):
            raise FieldError("Must be an ISO 8601 date, e.g. 2026-10-20 or 2026-10-20T18:00:00Z")
        return _check_range(value, rules)

    if kind == "list":
        if not isinstance(value, list):
            raise FieldError("Must be a list")
        if len(value) < rules.get("min_items", 0):
            raise FieldError(f"Must contain at least {rules['min_items']} item(s)")
        if "max_items" in rules and len(value) > rules["max_items"]:
            raise FieldError(f"Must contain at most {rules['max_items']} items")
        cleaned, errors = [], {}
        for i, item in enumerate(value):
```

```
            try:
                cleaned.append(check_value(item, rules.get("items", {})))
            except FieldError as e:
                errors[f"[{i}]"] = e.message
        if errors:
            raise FieldError(errors)
        return cleaned

    if kind == "object":
        if not isinstance(value, dict):
            raise FieldError("Must be an object")
        cleaned, errors = _validate(value, rules["schema"], rules.get("partial", False))
        if errors:
            raise FieldError(errors)
        return cleaned

    raise ValueError(f"Unknown rule type {kind}")      # programming error, not user error


def _validate(data, schema, partial):
    """Returns (cleaned_data, errors). partial=True skips 'required' checks (updates)."""
    cleaned, errors = {}, {}
    for key in data:
        if key not in schema:
            errors[key] = "Unknown field"
    for name, rules in schema.items():
        if name not in data:
            if rules.get("required") and not partial:
                errors[name] = "This field is required"
            continue
        try:
            cleaned[name] = check_value(data[name], rules)
        except FieldError as e:
            errors[name] = e.message
    return cleaned, errors


def to_dotted(data, prefix=""):
    """Flattens nested objects into MongoDB dot notation for partial $set updates.

    {"address": {"city": "Belfast"}} -> {"address.city": "Belfast"}
    so updating one sub-field does not wipe out the other sub-fields.
    """
    flat = {}
    for key, value in data.items():
        path = f"{prefix}{key}"
        if isinstance(value, dict) and value:
            flat.update(to_dotted(value, path + "."))
        else:
            flat[path] = value
    return flat


# ---------------------------------------------------------------- request helpers

def get_json_body():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        raise ApiError(400, "Request body must be a JSON object",
                       {"body": "Send a JSON object with the header Content-Type: application/json"}
  >>   )
    return data


def validate_body(schema, partial=False):
    """Validates the JSON body. partial=True is used by PUT routes that update some fields."""
    data = get_json_body()
    cleaned, errors = _validate(data, schema, partial)
    if partial and not cleaned and not errors:
        errors["body"] = "Provide at least one field to update"
    if errors:
        raise ApiError(400, "Validation failed", errors)
    return cleaned


def _convert_query_value(raw, rules):
    """Query string values are always text; convert them before the normal checks."""
    kind = rules.get("type", "string")
    if kind == "int":
        try:
```

```
            return int(raw)
        except ValueError:
            raise FieldError("Must be a whole number")
    if kind == "number":
        try:
            return float(raw)
        except ValueError:
            raise FieldError("Must be a number")
    if kind == "bool":
        if raw.lower() in ("true", "1", "yes"):
            return True
        if raw.lower() in ("false", "0", "no"):
            return False
        raise FieldError("Must be true or false")
    if kind == "list":                        # comma-separated values: ?level=Beginner,Expert
        return [part.strip() for part in raw.split(",") if part.strip()]
    return raw


def validate_query(schema):
    """Validates request.args against a schema and fills in defaults."""
    cleaned, errors = {}, {}
    for key in request.args:
        if key not in schema:
            errors[key] = "Unknown query parameter"
    for name, rules in schema.items():
        raw = request.args.get(name)
        if raw is None or raw.strip() == "":
            if rules.get("required"):
                errors[name] = "This query parameter is required"
            elif "default" in rules:
                cleaned[name] = rules["default"]
            continue
        try:
            cleaned[name] = check_value(_convert_query_value(raw.strip(), rules), rules)
        except FieldError as e:
            errors[name] = e.message
    if errors:
        raise ApiError(400, "Invalid query parameters", errors)
    return cleaned
```

utils/serialize.py

```
"""
JSON serialisation for MongoDB types.

Installing MongoJSONProvider on the app means every jsonify() call converts
ObjectId -> "hex string" and datetime -> "2026-10-12T18:00:00Z" automatically,
so routes can return documents straight from pymongo.
"""

from datetime import datetime, timezone

from bson import ObjectId
from flask.json.provider import DefaultJSONProvider


def to_iso(value):
    """Formats a datetime as ISO 8601 in UTC with a trailing Z."""
    if value.tzinfo is None:
        value = value.replace(tzinfo=timezone.utc)
    return value.astimezone(timezone.utc).isoformat(timespec="seconds").replace("+00:00", "Z")


class MongoJSONProvider(DefaultJSONProvider):
    sort_keys = False      # keep fields in the order they are stored / built

    @staticmethod
    def default(obj):
        if isinstance(obj, ObjectId):
            return str(obj)
        if isinstance(obj, datetime):
            return to_iso(obj)
        if isinstance(obj, bytes):          # never expected in output, but never crash on it
            return None
        return DefaultJSONProvider.default(obj)
```

utils/pagination.py

```
"""
Pagination helpers.

List endpoints accept ?pn=<page number>&ps=<page size> and return:
    {"data": [...], "pagination": {"page": 1, "page_size": 10, "total": 2909, "pages": 291}}
The total lets a front end draw page links without a second request.
"""

import math

# Merge into any query schema that supports paging
PAGINATION_QUERY = {
    "pn": {"type": "int", "min": 1, "default": 1},
    "ps": {"type": "int", "min": 1, "max": 100, "default": 10},
}


def page_envelope(data, total, pn, ps):
    return {
        "data": data,
        "pagination": {"page": pn, "page_size": ps, "total": total,
                       "pages": math.ceil(total / ps) if total else 0},
    }


def paginate_find(collection, query, pn, ps, projection=None, sort=None):
    """Runs a find() for one page and counts the full result set."""
    cursor = collection.find(query, projection)
    if sort:
        cursor = cursor.sort(sort)
    data = list(cursor.skip((pn - 1) * ps).limit(ps))
    return page_envelope(data, collection.count_documents(query), pn, ps)


def facet_page(pn, ps):
    """Aggregation stage that returns one page and the total count in a single query."""
    return {"$facet": {
        "data": [{"$skip": (pn - 1) * ps}, {"$limit": ps}],
        "total": [{"$count": "count"}],
    }}


def facet_envelope(result, pn, ps):
    """Converts the output of facet_page() into the standard envelope."""
    doc = result[0] if result else {"data": [], "total": []}
    total = doc["total"][0]["count"] if doc["total"] else 0
    return page_envelope(doc["data"], total, pn, ps)
```

utils/responses.py

```
"""Small helpers for success responses so every route answers in the same shape."""

from flask import jsonify, make_response, url_for


def created(message, new_id, endpoint, extra=None, **url_args):
    """201 Created with the new id, the URL of the new resource and a Location header.

    endpoint and url_args are passed to url_for, e.g.
        created("Review added", review_id, "reviews.get_review", id=ex_id, review_id=review_id)
    extra is an optional dict of additional fields for the response body.
    """
    url = url_for(endpoint, _external=True, **{k: str(v) for k, v in url_args.items()})
    body = {"message": message, "id": str(new_id), "url": url, **(extra or {})}
    response = make_response(jsonify(body), 201)
    response.headers["Location"] = url
    return response


def ok(body):
    return make_response(jsonify(body), 200)


def no_content():
    return make_response("", 204)
```

utils/constants.py

```
"""
Allowed values for enum fields. These match the values that exist in the seeded data,
so validation never rejects something the dataset itself contains.
"""

# Membership plan -> length of one period in days (same as seed.py)
PLAN_DAYS = {"Monthly": 30, "Quarterly": 90, "Annual": 365, "Student": 120}
MEMBERSHIP_STATUSES = ["active", "expired"]

GENDERS = ["Male", "Female"]
EXPERIENCE_LEVELS = ["Beginner", "Intermediate", "Expert"]
WORKOUT_TYPES = ["Cardio", "HIIT", "Strength", "Yoga"]      # also the class categories

CLASS_LEVELS = ["Beginner", "All levels", "Intermediate", "Advanced"]
CLASS_STATUSES = ["scheduled", "completed", "cancelled"]
BOOKING_STATUSES = ["booked", "cancelled"]

EXERCISE_TYPES = ["Cardio", "Olympic Weightlifting", "Plyometrics", "Powerlifting",
                  "Strength", "Stretching", "Strongman"]
BODY_PARTS = ["Abdominals", "Abductors", "Adductors", "Biceps", "Calves", "Chest", "Forearms",
              "Glutes", "Hamstrings", "Lats", "Lower Back", "Middle Back", "Neck", "Quadriceps",
              "Shoulders", "Traps", "Triceps"]
EQUIPMENT = ["Bands", "Barbell", "Body Only", "Cable", "Dumbbell", "E-Z Curl Bar", "Exercise Ball",
             "Foam Roll", "Kettlebells", "Machine", "Medicine Ball", "None", "Other"]
EXERCISE_LEVELS = ["Beginner", "Intermediate", "Expert"]

ROLES = ["admin", "trainer", "member"]
```

utils/membership.py

```
"""
Member profile rules shared by /auth/register and the members Blueprint.
"""

from datetime import timedelta

from utils.constants import EXPERIENCE_LEVELS, GENDERS, PLAN_DAYS, WORKOUT_TYPES
from utils.validation import now_utc

BODY_METRICS_SCHEMA = {
    "weight_kg": {"type": "number", "min": 30, "max": 300},
    "height_m": {"type": "number", "min": 1.0, "max": 2.5},
    "fat_percentage": {"type": "number", "min": 2, "max": 70},
    "resting_bpm": {"type": "int", "min": 30, "max": 120},
}

# Fields describing the person. Members may edit these on their own record.
PROFILE_SCHEMA = {
    "name": {"type": "string", "min_len": 2, "max_len": 80},
    "gender": {"type": "string", "choices": GENDERS},
    "age": {"type": "int", "min": 16, "max": 100},
    "home_branch_id": {"type": "objectid"},
    "body_metrics": {"type": "object", "schema": BODY_METRICS_SCHEMA, "partial": True},
    "preferred_workout": {"type": "string", "choices": WORKOUT_TYPES},
    "experience_level": {"type": "string", "choices": EXPERIENCE_LEVELS},
    "workout_frequency": {"type": "int", "min": 0, "max": 7},
    "water_intake_l": {"type": "number", "min": 0, "max": 10},
}

REQUIRED_PROFILE_FIELDS = ["name", "gender", "age", "home_branch_id"]


def compute_bmi(weight_kg, height_m):
    if weight_kg is None or height_m is None:
        return None
    return round(weight_kg / (height_m ** 2), 1)


def new_membership(plan, start=None):
    """A membership period starting today (or at start) for the plan's number of days."""
    start = start or now_utc().replace(hour=0, minute=0, second=0, microsecond=0)
    return {"plan": plan, "start_date": start,
            "end_date": start + timedelta(days=PLAN_DAYS[plan]), "status": "active"}


def new_member_document(data):
    """Builds a member document with the same shape as the seeded ones."""
    metrics = data.get("body_metrics", {})
    weight, height = metrics.get("weight_kg"), metrics.get("height_m")
    return {
        "name": data["name"],
        "email": data["email"],
        "gender": data["gender"],
        "age": data["age"],
        "home_branch_id": data["home_branch_id"],
        "body_metrics": {
            "weight_kg": weight,
            "height_m": height,
            "bmi": compute_bmi(weight, height),
            "fat_percentage": metrics.get("fat_percentage"),
            "resting_bpm": metrics.get("resting_bpm"),
        },
        "preferred_workout": data.get("preferred_workout"),
        "experience_level": data.get("experience_level", "Beginner"),
        "workout_frequency": data.get("workout_frequency"),
        "water_intake_l": data.get("water_intake_l"),
        "membership": new_membership(data.get("plan", "Monthly")),
        "workouts": [],
        "joined_at": now_utc(),
    }
```

blueprints/auth/auth.py

```
"""
auth Blueprint - registration, login (JWT), logout (token blacklist) and the current user.

    POST /api/v1.0/auth/register   create a user account and its linked member record
    POST /api/v1.0/auth/login      HTTP Basic auth -> JWT valid for 30 minutes
    GET  /api/v1.0/auth/logout     blacklist the current token
    GET  /api/v1.0/auth/me         the logged-in user's account (and member summary)
"""

from datetime import datetime, timedelta, timezone

import bcrypt
import jwt
from bson import ObjectId
from flask import Blueprint, g, request

from decorators import jwt_required
from globals import SECRET_KEY, TOKEN_EXPIRY_MINUTES, blacklist, branches, members, users
from utils.constants import PLAN_DAYS
from utils.errors import ApiError
from utils.membership import PROFILE_SCHEMA, REQUIRED_PROFILE_FIELDS, new_member_document
from utils.responses import created, ok
from utils.validation import now_utc, validate_body

auth_bp = Blueprint("auth", __name__)

# Registration = account fields + the member profile fields (required ones marked here)
REGISTER_SCHEMA = {
    "username": {"type": "string", "required": True, "min_len": 3, "max_len": 30,
                 "pattern": r"[A-Za-z0-9._-]+",
                 "pattern_msg": "Use only letters, numbers, dots, underscores and hyphens"},
    "email": {"type": "email", "required": True, "max_len": 120},
    # bcrypt only uses the first 72 bytes of a password, so longer ones are rejected
    "password": {"type": "string", "required": True, "min_len": 8, "max_len": 72},
    "plan": {"type": "string", "choices": list(PLAN_DAYS)},
    **{name: {**rules, "required": name in REQUIRED_PROFILE_FIELDS}
       for name, rules in PROFILE_SCHEMA.items()},
}


def _public_user(user):
    """A user document without the password hash."""
    user = dict(user)
    user.pop("password", None)
    return user


@auth_bp.route("/auth/register", methods=["POST"])
def register():
    data = validate_body(REGISTER_SCHEMA)
    data["username"] = data["username"].lower()

    # Check every conflict first so the user sees all of them at once
    conflicts = {}
    if users.find_one({"username": data["username"]}):
        conflicts["username"] = "This username is already taken"
    if users.find_one({"email": data["email"]}) or members.find_one({"email": data["email"]}):
        conflicts["email"] = "An account with this email already exists"
    if conflicts:
        raise ApiError(409, "Account already exists", conflicts)
    if not branches.find_one({"_id": data["home_branch_id"]}, {"_id": 1}):
        raise ApiError(400, "Validation failed", {"home_branch_id": "No branch exists with this id"}
 >>   )

    member_id = members.insert_one(new_member_document(data)).inserted_id
    try:
        user_id = users.insert_one({
            "username": data["username"],
            "email": data["email"],
            "name": data["name"],
            "password": bcrypt.hashpw(data["password"].encode(), bcrypt.gensalt()),
            "role": "member",
            "member_id": member_id,
            "created_at": now_utc(),
        }).inserted_id
    except Exception:
        # Two writes without a transaction: undo the first if the second fails
        members.delete_one({"_id": member_id})
```

```
        raise

    # The new member record is the useful resource to link to; the user id is returned too
    return created("Account created", member_id, "members.get_member",
                   extra={"user_id": str(user_id)}, id=member_id)


@auth_bp.route("/auth/login", methods=["POST"])
def login():
    auth = request.authorization
    if not auth or not auth.username or not auth.password:
        raise ApiError(401, "Login details are missing",
                       {"authorization": "Send the username and password using HTTP Basic auth"})

    user = users.find_one({"username": auth.username.lower()})
    # Same message for unknown user and wrong password, so usernames can't be probed
    if not user or not bcrypt.checkpw(auth.password.encode(), bytes(user["password"])):
        raise ApiError(401, "Invalid username or password")

    expires = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_EXPIRY_MINUTES)
    payload = {
        "user_id": str(user["_id"]),
        "username": user["username"],
        "role": user["role"],
        "member_id": str(user["member_id"]) if user.get("member_id") else None,
        "exp": expires,
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")
    return ok({
        "token": token,
        "expires_at": expires,
        "user": {k: v for k, v in payload.items() if k != "exp"},
    })


@auth_bp.route("/auth/logout", methods=["GET"])
@jwt_required
def logout():
    # Store the token until it would have expired anyway; a TTL index on expires_at
    # (created in app.py) lets MongoDB delete old entries automatically.
    blacklist.insert_one({
        "token": g.token,
        "expires_at": datetime.fromtimestamp(g.user["exp"], tz=timezone.utc),
        "logged_out_at": now_utc(),
    })
    return ok({"message": "Logged out successfully"})


@auth_bp.route("/auth/me", methods=["GET"])
@jwt_required
def me():
    user = users.find_one({"_id": ObjectId(g.user["user_id"])})
    if not user:
        raise ApiError(404, "User account no longer exists")
    body = _public_user(user)
    if user.get("member_id"):
        body["member"] = members.find_one(
            {"_id": user["member_id"]},
            {"name": 1, "home_branch_id": 1, "membership": 1, "preferred_workout": 1,
             "experience_level": 1})
    return ok(body)
```

blueprints/branches/branches.py

```
"""
branches Blueprint - gym branches, nearest-branch search and each branch's timetable.

    GET    /api/v1.0/branches                 list (filter by city, facility, name search)
    POST   /api/v1.0/branches                 create                      (admin)
    GET    /api/v1.0/branches/near            branches within km of a point ($geoNear)
    GET    /api/v1.0/branches/<id>            one branch
    PUT    /api/v1.0/branches/<id>            update some fields          (admin)
    DELETE /api/v1.0/branches/<id>            delete if nothing uses it   (admin)
    GET    /api/v1.0/branches/<id>/classes    upcoming classes at the branch
"""

import re

from flask import Blueprint

from decorators import role_required
from globals import branches, classes, members, users
from utils.constants import WORKOUT_TYPES
from utils.errors import ApiError
from utils.pagination import PAGINATION_QUERY, paginate_find
from utils.responses import created, no_content, ok
from utils.validation import now_utc, parse_object_id, to_dotted, validate_body, validate_query

branches_bp = Blueprint("branches", __name__)

HOURS_RULE = {"type": "string", "required": True, "pattern": r"([01]\d|2[0-3]):[0-5]\d-([01]\d|2[0-3
>>   ]):[0-5]\d",
              "pattern_msg": "Use the format HH:MM-HH:MM, e.g. 06:00-22:00"}

BRANCH_SCHEMA = {
    "name": {"type": "string", "required": True, "min_len": 3, "max_len": 80},
    "address": {"type": "object", "required": True, "schema": {
        "area": {"type": "string", "required": True, "max_len": 80},
        "city": {"type": "string", "required": True, "max_len": 80},
        "postcode": {"type": "string", "required": True, "max_len": 8,
                     "pattern": r"[A-Za-z]{1,2}\d[A-Za-z\d]?(\s?\d[A-Za-z]{2})?",
                     "pattern_msg": "Must be a UK postcode or postcode district, e.g. BT1 or BT1 5GS
>>   "},
    }},
    "lat": {"type": "number", "required": True, "min": -90, "max": 90},
    "lng": {"type": "number", "required": True, "min": -180, "max": 180},
    "opening_hours": {"type": "object", "required": True, "schema": {
        "weekdays": HOURS_RULE, "weekends": HOURS_RULE}},
    "facilities": {"type": "list", "max_items": 30, "items": {"type": "string", "max_len": 50}},
    "capacity": {"type": "int", "required": True, "min": 1, "max": 1000},
    "phone": {"type": "string", "required": True, "pattern": r"[0-9 +()-]{7,20}",
              "pattern_msg": "Must be 7-20 digits, spaces or + ( ) -"},
}

# For updates the nested objects may also be partial (e.g. just address.city)
UPDATE_SCHEMA = {**BRANCH_SCHEMA,
                 "address": {**BRANCH_SCHEMA["address"], "partial": True},
                 "opening_hours": {**BRANCH_SCHEMA["opening_hours"], "partial": True}}


def _find_branch(branch_id):
    branch = branches.find_one({"_id": parse_object_id(branch_id, "branch id")})
    if not branch:
        raise ApiError(404, "Branch not found")
    return branch


def _name_taken(name, exclude_id=None):
    query = {"name": {"$regex": f"^{re.escape(name)}$", "$options": "i"}}
    if exclude_id:
        query["_id"] = {"$ne": exclude_id}
    return branches.find_one(query, {"_id": 1}) is not None


def _prepare(data):
    """Turns validated input into document fields: lat/lng become a GeoJSON Point."""
    if "lat" in data or "lng" in data:
        if "lat" not in data or "lng" not in data:
            raise ApiError(400, "Validation failed", {"lat/lng": "Send lat and lng together"})
        # GeoJSON order is [longitude, latitude]
        data["location"] = {"type": "Point", "coordinates": [data.pop("lng"), data.pop("lat")]}
    if "postcode" in data.get("address", {}):
```

```
        data["address"]["postcode"] = data["address"]["postcode"].upper()
    return data


@branches_bp.route("/branches", methods=["GET"])
def list_branches():
    args = validate_query({
        "city": {"type": "string", "max_len": 80},
        "facility": {"type": "list", "items": {"type": "string"}},
        "q": {"type": "string", "max_len": 80},
        **PAGINATION_QUERY,
    })
    query = {}
    if "city" in args:
        query["address.city"] = {"$regex": f"^{re.escape(args['city'])}$", "$options": "i"}
    if "facility" in args:
        # every requested facility must be present (case-insensitive)
        query["facilities"] = {"$all": [re.compile(f"^{re.escape(f)}$", re.I) for f in args["facilit
 >>   y"]]}
    if "q" in args:
        pattern = {"$regex": re.escape(args["q"]), "$options": "i"}
        query["$or"] = [{"name": pattern}, {"address.area": pattern}]
    return ok(paginate_find(branches, query, args["pn"], args["ps"], sort=[("name", 1)]))


@branches_bp.route("/branches", methods=["POST"])
@role_required("admin")
def create_branch():
    data = _prepare(validate_body(BRANCH_SCHEMA))
    if _name_taken(data["name"]):
        raise ApiError(409, "A branch with this name already exists")
    data.setdefault("facilities", [])
    data["created_at"] = now_utc()
    new_id = branches.insert_one(data).inserted_id
    return created("Branch created", new_id, "branches.get_branch", id=new_id)


@branches_bp.route("/branches/near", methods=["GET"])
def branches_near():
    args = validate_query({
        "lat": {"type": "number", "required": True, "min": -90, "max": 90},
        "lng": {"type": "number", "required": True, "min": -180, "max": 180},
        "km": {"type": "number", "min": 0, "max": 100, "default": 10},
        "limit": {"type": "int", "min": 1, "max": 50, "default": 10},
    })
    pipeline = [
        # $geoNear must be the first stage; it uses the 2dsphere index on location
        # and returns documents nearest first, adding the distance in metres.
        {"$geoNear": {
            "near": {"type": "Point", "coordinates": [args["lng"], args["lat"]]},
            "distanceField": "distance_m",
            "maxDistance": args["km"] * 1000,
            "spherical": True,
        }},
        {"$limit": args["limit"]},
        {"$addFields": {"distance_km": {"$round": [{"$divide": ["$distance_m", 1000]}, 2]}}},
        {"$project": {"distance_m": 0}},
    ]
    results = list(branches.aggregate(pipeline))
    return ok({"data": results, "count": len(results),
               "search": {"lat": args["lat"], "lng": args["lng"], "km": args["km"]}})


@branches_bp.route("/branches/<id>", methods=["GET"])
def get_branch(id):
    return ok(_find_branch(id))


@branches_bp.route("/branches/<id>", methods=["PUT"])
@role_required("admin")
def update_branch(id):
    branch = _find_branch(id)
    data = _prepare(validate_body(UPDATE_SCHEMA, partial=True))
    if "name" in data and _name_taken(data["name"], exclude_id=branch["_id"]):
        raise ApiError(409, "A branch with this name already exists")

    # location and facilities are replaced whole; address / opening_hours per sub-field
    location = data.pop("location", None)
    updates = to_dotted(data)
    if location:
```

```
        updates["location"] = location
    branches.update_one({"_id": branch["_id"]}, {"$set": updates})

    # Classes keep a copy of the branch name, so keep those copies in step
    if "name" in data:
        classes.update_many({"branch_id": branch["_id"]}, {"$set": {"branch_name": data["name"]}})
    return ok({"message": "Branch updated", "branch": branches.find_one({"_id": branch["_id"]})})


@branches_bp.route("/branches/<id>", methods=["DELETE"])
@role_required("admin")
def delete_branch(id):
    branch = _find_branch(id)
    in_use = {
        "members": members.count_documents({"home_branch_id": branch["_id"]}),
        "trainers": users.count_documents({"role": "trainer", "branch_id": branch["_id"]}),
        "upcoming_classes": classes.count_documents(
            {"branch_id": branch["_id"], "status": "scheduled", "start_time": {"$gt": now_utc()}}),
    }
    if any(in_use.values()):
        raise ApiError(409, "Branch is still in use and cannot be deleted",
                       {k: f"{v} linked" for k, v in in_use.items() if v})
    branches.delete_one({"_id": branch["_id"]})
    return no_content()


@branches_bp.route("/branches/<id>/classes", methods=["GET"])
def branch_classes(id):
    branch = _find_branch(id)
    args = validate_query({
        "category": {"type": "list", "items": {"type": "string", "choices": WORKOUT_TYPES}},
        **PAGINATION_QUERY,
    })
    query = {"branch_id": branch["_id"], "status": "scheduled", "start_time": {"$gt": now_utc()}}
    if "category" in args:
        query["category"] = {"$in": args["category"]}
    page = paginate_find(classes, query, args["pn"], args["ps"],
                         projection={"bookings": 0}, sort=[("start_time", 1)])
    for c in page["data"]:
        c["spaces_left"] = c["capacity"] - c["booked_count"]
    return ok({"branch": {"_id": branch["_id"], "name": branch["name"]}, **page})
```

blueprints/exercises/exercises.py

```
"""
exercises Blueprint - the exercise library.

    GET    /api/v1.0/exercises               search, filter, sort and paginate
    POST   /api/v1.0/exercises               create              (admin, trainer)
    GET    /api/v1.0/exercises/filters       distinct values (with counts) for each filter
    GET    /api/v1.0/exercises/top-rated     best average review stars (aggregation)
    GET    /api/v1.0/exercises/<id>          one exercise with its reviews
    PUT    /api/v1.0/exercises/<id>          update some fields  (admin, trainer)
    DELETE /api/v1.0/exercises/<id>          delete              (admin, trainer)
"""

import re

from flask import Blueprint

from decorators import role_required
from globals import exercises
from utils.constants import BODY_PARTS, EQUIPMENT, EXERCISE_LEVELS, EXERCISE_TYPES
from utils.errors import ApiError
from utils.pagination import PAGINATION_QUERY, facet_envelope, facet_page
from utils.responses import created, no_content, ok
from utils.validation import parse_object_id, validate_body, validate_query

exercises_bp = Blueprint("exercises", __name__)

EXERCISE_SCHEMA = {
    "title": {"type": "string", "required": True, "min_len": 3, "max_len": 120},
    "description": {"type": "string", "nullable": True, "max_len": 2000},
    "type": {"type": "string", "required": True, "choices": EXERCISE_TYPES},
    "body_part": {"type": "string", "required": True, "choices": BODY_PARTS},
    "equipment": {"type": "string", "required": True, "choices": EQUIPMENT},
    "level": {"type": "string", "required": True, "choices": EXERCISE_LEVELS},
    "rating": {"type": "number", "nullable": True, "min": 0, "max": 10},
}

# Filters that accept comma-separated values, e.g. ?body_part=Chest,Triceps
LIST_FILTERS = {"body_part": BODY_PARTS, "equipment": EQUIPMENT, "level": EXERCISE_LEVELS,
                "type": EXERCISE_TYPES}


def find_exercise(exercise_id, projection=None):
    """Shared with the reviews Blueprint."""
    exercise = exercises.find_one({"_id": parse_object_id(exercise_id, "exercise id")}, projection)
    if not exercise:
        raise ApiError(404, "Exercise not found")
    return exercise


def _title_taken(title, exclude_id=None):
    query = {"title": {"$regex": f"^{re.escape(title)}$", "$options": "i"}}
    if exclude_id:
        query["_id"] = {"$ne": exclude_id}
    return exercises.find_one(query, {"_id": 1}) is not None


@exercises_bp.route("/exercises", methods=["GET"])
def list_exercises():
    args = validate_query({
        **{name: {"type": "list", "items": {"type": "string", "choices": values}}
           for name, values in LIST_FILTERS.items()},
        "min_rating": {"type": "number", "min": 0, "max": 10},
        "q": {"type": "string", "max_len": 100},
        "sort": {"type": "string", "choices": ["rating", "title", "reviews"]},
        "order": {"type": "string", "choices": ["asc", "desc"]},
        **PAGINATION_QUERY,
    })

    match = {}
    if "q" in args:
        match["$text"] = {"$search": args["q"]}          # uses the text index on title + descriptio
>>  n
    for field in LIST_FILTERS:
        if field in args:
            match[field] = {"$in": args[field]}
    if "min_rating" in args:
        match["rating"] = {"$gte": args["min_rating"]}

```

```
    # Default order: best text match when searching, otherwise alphabetical
    sort_field = args.get("sort") or ("relevance" if "q" in args else "title")
    default_dir = "asc" if sort_field == "title" else "desc"
    direction = 1 if args.get("order", default_dir) == "asc" else -1
    sort = {
        "relevance": {"score": {"$meta": "textScore"}},
        "rating": {"rating": direction, "title": 1},
        "title": {"title": direction},
        "reviews": {"review_count": direction, "title": 1},
    }[sort_field]

    pipeline = [
        {"$match": match},
        {"$addFields": {"review_count": {"$size": "$reviews"},
                        "avg_stars": {"$round": [{"$avg": "$reviews.stars"}, 2]}}},
        {"$project": {"reviews": 0}},
        {"$sort": sort},
        facet_page(args["pn"], args["ps"]),
    ]
    # Case-insensitive ordering of titles ("bench" next to "Bench"). A $text search
    # cannot be combined with a collation, so it keeps the default.
    options = {} if "q" in args else {"collation": {"locale": "en", "strength": 2}}
    result = list(exercises.aggregate(pipeline, **options))
    return ok(facet_envelope(result, args["pn"], args["ps"]))


@exercises_bp.route("/exercises", methods=["POST"])
@role_required("admin", "trainer")
def create_exercise():
    data = validate_body(EXERCISE_SCHEMA)
    if _title_taken(data["title"]):
        raise ApiError(409, "An exercise with this title already exists")
    data.setdefault("description", None)
    data.setdefault("rating", None)
    data["reviews"] = []
    new_id = exercises.insert_one(data).inserted_id
    return created("Exercise created", new_id, "exercises.get_exercise", id=new_id)


@exercises_bp.route("/exercises/filters", methods=["GET"])
def exercise_filters():
    # One query: $facet runs a $sortByCount for each field in parallel
    pipeline = [{"$facet": {field: [{"$sortByCount": f"${field}"},
                                    {"$project": {"_id": 0, "value": "$_id", "count": 1}}]
                            for field in LIST_FILTERS}}]
    return ok(list(exercises.aggregate(pipeline))[0])


@exercises_bp.route("/exercises/top-rated", methods=["GET"])
def top_rated():
    args = validate_query({
        "body_part": {"type": "list", "items": {"type": "string", "choices": BODY_PARTS}},
        "min_reviews": {"type": "int", "min": 1, "max": 50, "default": 2},
        "limit": {"type": "int", "min": 1, "max": 50, "default": 10},
    })
    match = {"reviews.0": {"$exists": True}}         # at least one review
    if "body_part" in args:
        match["body_part"] = {"$in": args["body_part"]}
    pipeline = [
        {"$match": match},
        {"$project": {
            "title": 1, "body_part": 1, "equipment": 1, "level": 1, "type": 1,
            "review_count": {"$size": "$reviews"},
            "avg_stars": {"$avg": "$reviews.stars"},
        }},
        # Ignore exercises with too few reviews: one 5-star review is not "top rated"
        {"$match": {"review_count": {"$gte": args["min_reviews"]}}},
        {"$sort": {"avg_stars": -1, "review_count": -1, "title": 1}},
        {"$limit": args["limit"]},
        {"$set": {"avg_stars": {"$round": ["$avg_stars", 2]}}},
    ]
    results = list(exercises.aggregate(pipeline))
    return ok({"data": results, "count": len(results), "min_reviews": args["min_reviews"]})


@exercises_bp.route("/exercises/<id>", methods=["GET"])
def get_exercise(id):
    exercise = find_exercise(id)
    stars = [r["stars"] for r in exercise["reviews"]]
    exercise["review_count"] = len(stars)
```

```
158      exercise["avg_stars"] = round(sum(stars) / len(stars), 2) if stars else None
159      return ok(exercise)
160
161
162  @exercises_bp.route("/exercises/<id>", methods=["PUT"])
163  @role_required("admin", "trainer")
164  def update_exercise(id):
165      exercise = find_exercise(id, {"_id": 1})
166      data = validate_body(EXERCISE_SCHEMA, partial=True)
167      if "title" in data and _title_taken(data["title"], exclude_id=exercise["_id"]):
168          raise ApiError(409, "An exercise with this title already exists")
169      exercises.update_one({"_id": exercise["_id"]}, {"$set": data})
170      return ok({"message": "Exercise updated",
171                 "exercise": exercises.find_one({"_id": exercise["_id"]}, {"reviews": 0})})
172
173
174  @exercises_bp.route("/exercises/<id>", methods=["DELETE"])
175  @role_required("admin", "trainer")
176  def delete_exercise(id):
177      exercise = find_exercise(id, {"_id": 1})
178      exercises.delete_one({"_id": exercise["_id"]})
179      # Workout logs keep their own copy of the title, so past workouts stay readable
180      return no_content()
```

blueprints/reviews/reviews.py

```
"""
reviews Blueprint - member reviews embedded in each exercise document (sub-documents).

    GET    /api/v1.0/exercises/<id>/reviews                all reviews + average
    POST   /api/v1.0/exercises/<id>/reviews                add a review          (member)
    GET    /api/v1.0/exercises/<id>/reviews/<review_id>    one review
    PUT    /api/v1.0/exercises/<id>/reviews/<review_id>    edit                  (review author)
    DELETE /api/v1.0/exercises/<id>/reviews/<review_id>    delete                (author or admin)
"""

from bson import ObjectId
from flask import Blueprint, g

from blueprints.exercises.exercises import find_exercise
from decorators import current_member_id, is_admin, jwt_required
from globals import exercises, members
from utils.errors import ApiError
from utils.responses import created, no_content, ok
from utils.validation import now_utc, parse_object_id, validate_body, validate_query

reviews_bp = Blueprint("reviews", __name__)

REVIEW_SCHEMA = {
    "stars": {"type": "int", "required": True, "min": 1, "max": 5},
    "comment": {"type": "string", "required": True, "min_len": 2, "max_len": 500},
}


def _find_review(exercise_id, review_id):
    """Returns (exercise_id, review). 404 tells apart a missing exercise and a missing review."""
    exercise = find_exercise(exercise_id, {"_id": 1})
    rid = parse_object_id(review_id, "review id")
    # The positional projection "reviews.$" returns only the matching array element
    doc = exercises.find_one({"_id": exercise["_id"], "reviews._id": rid}, {"reviews.$": 1})
    if not doc:
        raise ApiError(404, "Review not found")
    return exercise["_id"], doc["reviews"][0]


def _ensure_author(review):
    if g.user.get("member_id") != str(review["member_id"]):
        raise ApiError(403, "You can only change your own reviews")


@reviews_bp.route("/exercises/<id>/reviews", methods=["GET"])
def list_reviews(id):
    args = validate_query({"sort": {"type": "string", "choices": ["newest", "oldest", "stars"],
                                    "default": "newest"}})
    exercise = find_exercise(id, {"title": 1, "reviews": 1})
    reviews = exercise["reviews"]
    if args["sort"] == "stars":
        reviews.sort(key=lambda r: (r["stars"], r["created_at"]), reverse=True)
    else:
        reviews.sort(key=lambda r: r["created_at"], reverse=args["sort"] == "newest")
    stars = [r["stars"] for r in reviews]
    return ok({
        "exercise": {"_id": exercise["_id"], "title": exercise["title"]},
        "data": reviews,
        "count": len(reviews),
        "avg_stars": round(sum(stars) / len(stars), 2) if stars else None,
    })


@reviews_bp.route("/exercises/<id>/reviews", methods=["POST"])
@jwt_required
def add_review(id):
    member_id = current_member_id()                # staff accounts can't review
    exercise = find_exercise(id, {"_id": 1})
    data = validate_body(REVIEW_SCHEMA)
    member = members.find_one({"_id": member_id}, {"name": 1})
    if not member:
        raise ApiError(404, "Your member record no longer exists")

    review = {"_id": ObjectId(), "member_id": member_id, "member_name": member["name"],
              "stars": data["stars"], "comment": data["comment"], "created_at": now_utc()}
    # One review per member per exercise. The check is part of the update filter,
    # so two requests at the same moment cannot both add a review.
    result = exercises.update_one(
```

```
        {"_id": exercise["_id"], "reviews.member_id": {"$ne": member_id}},
        {"$push": {"reviews": review}},
    )
    if result.modified_count == 0:
        raise ApiError(409, "You have already reviewed this exercise",
                       {"hint": "Edit your existing review with PUT instead"})
    return created("Review added", review["_id"], "reviews.get_review",
                   id=exercise["_id"], review_id=review["_id"])


@reviews_bp.route("/exercises/<id>/reviews/<review_id>", methods=["GET"])
def get_review(id, review_id):
    return ok(_find_review(id, review_id)[1])


@reviews_bp.route("/exercises/<id>/reviews/<review_id>", methods=["PUT"])
@jwt_required
def update_review(id, review_id):
    exercise_id, review = _find_review(id, review_id)
    _ensure_author(review)
    data = validate_body(REVIEW_SCHEMA, partial=True)
    # "reviews.$" is the positional operator: it updates the element matched by reviews._id
    updates = {f"reviews.$.{field}": value for field, value in data.items()}
    updates["reviews.$.updated_at"] = now_utc()
    exercises.update_one({"_id": exercise_id, "reviews._id": review["_id"]}, {"$set": updates})
    return ok({"message": "Review updated", "review": _find_review(id, review_id)[1]})


@reviews_bp.route("/exercises/<id>/reviews/<review_id>", methods=["DELETE"])
@jwt_required
def delete_review(id, review_id):
    exercise_id, review = _find_review(id, review_id)
    if not is_admin():                      # admins can moderate any review
        _ensure_author(review)
    exercises.update_one({"_id": exercise_id}, {"$pull": {"reviews": {"_id": review["_id"]}}})
    return no_content()
```

blueprints/members/members.py

```
"""
members Blueprint - member records, memberships, progress statistics and bookings.

    GET    /api/v1.0/members                    list with filters             (admin)
    POST   /api/v1.0/members                    create a member record        (admin)
    GET    /api/v1.0/members/<id>               one member                    (admin or self)
    PUT    /api/v1.0/members/<id>               update profile fields         (admin or self)
    DELETE /api/v1.0/members/<id>               delete member + account       (admin)
    PUT    /api/v1.0/members/<id>/membership    renew or change plan          (admin)
    GET    /api/v1.0/members/<id>/stats         workout statistics            (admin or self)
    GET    /api/v1.0/members/<id>/bookings      class bookings across classes (admin or self)
"""

import re
from datetime import timedelta

from flask import Blueprint, request

from decorators import ensure_self_or_admin, is_admin, jwt_required, role_required
from globals import branches, classes, exercises, members, users
from utils.constants import BOOKING_STATUSES, GENDERS, MEMBERSHIP_STATUSES, PLAN_DAYS, WORKOUT_TYPES
from utils.errors import ApiError
from utils.membership import (PROFILE_SCHEMA, REQUIRED_PROFILE_FIELDS, compute_bmi,
                              new_member_document, new_membership)
from utils.pagination import PAGINATION_QUERY, facet_envelope, facet_page
from utils.responses import created, no_content, ok
from utils.validation import now_utc, parse_object_id, to_dotted, validate_body, validate_query

members_bp = Blueprint("members", __name__)

# Admins can create members (e.g. at the front desk) without a login account
CREATE_SCHEMA = {
    "email": {"type": "email", "required": True, "max_len": 120},
    "plan": {"type": "string", "choices": list(PLAN_DAYS)},
    **{name: {**rules, "required": name in REQUIRED_PROFILE_FIELDS}
       for name, rules in PROFILE_SCHEMA.items()},
}
ADMIN_UPDATE_SCHEMA = {"email": CREATE_SCHEMA["email"], **PROFILE_SCHEMA}

# Fields left out of member lists and single-member responses (workouts have their own endpoint)
SUMMARY_PROJECTION = {"workouts": 0}


def find_member(member_id, projection=None):
    """Access check first, then lookup - shared with the workouts Blueprint.

    Checking access before existence means a member probing other ids always gets 403
    and never learns which ids exist.
    """
    oid = parse_object_id(member_id, "member id")
    ensure_self_or_admin(oid)
    member = members.find_one({"_id": oid}, projection)
    if not member:
        raise ApiError(404, "Member not found")
    return member


def _check_branch(branch_id):
    if not branches.find_one({"_id": branch_id}, {"_id": 1}):
        raise ApiError(400, "Validation failed", {"home_branch_id": "No branch exists with this id"}
 >>   )


def _email_taken(email, member_id=None):
    member_query = {"email": email}
    user_query = {"email": email}
    if member_id:
        member_query["_id"] = {"$ne": member_id}
        user_query["member_id"] = {"$ne": member_id}
    return members.find_one(member_query, {"_id": 1}) or users.find_one(user_query, {"_id": 1})


@members_bp.route("/members", methods=["GET"])
@role_required("admin")
def list_members():
    args = validate_query({
        "branch": {"type": "objectid"},
        "status": {"type": "string", "choices": MEMBERSHIP_STATUSES},
```

```
        "plan": {"type": "list", "items": {"type": "string", "choices": list(PLAN_DAYS)}},
        "workout_type": {"type": "list", "items": {"type": "string", "choices": WORKOUT_TYPES}},
        "gender": {"type": "string", "choices": GENDERS},
        "q": {"type": "string", "max_len": 80},
        "sort": {"type": "string", "choices": ["name", "joined_at", "age", "end_date"], "default": "
 >>   name"},
        **PAGINATION_QUERY,
    })
    match = {}
    if "branch" in args:
        match["home_branch_id"] = args["branch"]
    if "status" in args:
        # Work the status out from end_date so it is always correct today,
        # even if the stored status field has not been refreshed.
        op = "$gte" if args["status"] == "active" else "$lt"
        match["membership.end_date"] = {op: now_utc()}
    if "plan" in args:
        match["membership.plan"] = {"$in": args["plan"]}
    if "workout_type" in args:
        match["preferred_workout"] = {"$in": args["workout_type"]}
    if "gender" in args:
        match["gender"] = args["gender"]
    if "q" in args:
        pattern = {"$regex": re.escape(args["q"]), "$options": "i"}
        match["$or"] = [{"name": pattern}, {"email": pattern}]

    sort_field = {"name": "name", "joined_at": "joined_at", "age": "age",
                  "end_date": "membership.end_date"}[args["sort"]]
    pipeline = [
        {"$match": match},
        {"$addFields": {"workout_count": {"$size": "$workouts"}}},
        {"$project": SUMMARY_PROJECTION},
        {"$sort": {sort_field: 1, "_id": 1}},
        facet_page(args["pn"], args["ps"]),
    ]
    return ok(facet_envelope(list(members.aggregate(pipeline)), args["pn"], args["ps"]))


@members_bp.route("/members", methods=["POST"])
@role_required("admin")
def create_member():
    data = validate_body(CREATE_SCHEMA)
    if _email_taken(data["email"]):
        raise ApiError(409, "A member with this email already exists")
    _check_branch(data["home_branch_id"])
    new_id = members.insert_one(new_member_document(data)).inserted_id
    return created("Member created", new_id, "members.get_member", id=new_id)


@members_bp.route("/members/<id>", methods=["GET"])
@jwt_required
def get_member(id):
    member = find_member(id)
    member["workout_count"] = len(member.pop("workouts"))
    branch = branches.find_one({"_id": member["home_branch_id"]}, {"name": 1})
    member["home_branch"] = branch
    return ok(member)


@members_bp.route("/members/<id>", methods=["PUT"])
@jwt_required
def update_member(id):
    member = find_member(id, SUMMARY_PROJECTION)
    if not is_admin() and "email" in (request.get_json(silent=True) or {}):
        raise ApiError(403, "Only an admin can change a member's email address")
    data = validate_body(ADMIN_UPDATE_SCHEMA if is_admin() else PROFILE_SCHEMA, partial=True)

    if "home_branch_id" in data:
        _check_branch(data["home_branch_id"])
    if "email" in data and _email_taken(data["email"], member["_id"]):
        raise ApiError(409, "Another account already uses this email")

    updates = to_dotted(data)
    if "body_metrics" in data:
        # Recalculate BMI from the new values merged with the stored ones
        metrics = {**member["body_metrics"], **data["body_metrics"]}
        updates["body_metrics.bmi"] = compute_bmi(metrics.get("weight_kg"), metrics.get("height_m"))
    members.update_one({"_id": member["_id"]}, {"$set": updates})

    # The member's name and email are copied into other collections; keep them in step.
```

```
    if "email" in data:
        users.update_one({"member_id": member["_id"]}, {"$set": {"email": data["email"]}})
    if "name" in data:
        users.update_one({"member_id": member["_id"]}, {"$set": {"name": data["name"]}})
        # array_filters update only the array elements that belong to this member
        classes.update_many({"bookings.member_id": member["_id"]},
                            {"$set": {"bookings.$[b].member_name": data["name"]}},
                            array_filters=[{"b.member_id": member["_id"]}])
        exercises.update_many({"reviews.member_id": member["_id"]},
                              {"$set": {"reviews.$[r].member_name": data["name"]}},
                              array_filters=[{"r.member_id": member["_id"]}])
    return ok({"message": "Member updated",
               "member": members.find_one({"_id": member["_id"]}, SUMMARY_PROJECTION)})


@members_bp.route("/members/<id>", methods=["DELETE"])
@role_required("admin")
def delete_member(id):
    member = find_member(id, {"_id": 1})
    mid = member["_id"]
    # Free their places in upcoming classes: mark the booking cancelled and decrement
    # booked_count in the same update so the count always matches the bookings.
    classes.update_many(
        {"status": "scheduled", "start_time": {"$gt": now_utc()},
         "bookings": {"$elemMatch": {"member_id": mid, "status": "booked"}}},
        {"$set": {"bookings.$[b].status": "cancelled"}, "$inc": {"booked_count": -1}},
        array_filters=[{"b.member_id": mid, "b.status": "booked"}],
    )
    exercises.update_many({"reviews.member_id": mid}, {"$pull": {"reviews": {"member_id": mid}}})
    users.delete_one({"member_id": mid})
    members.delete_one({"_id": mid})
    return no_content()


@members_bp.route("/members/<id>/membership", methods=["PUT"])
@role_required("admin")
def update_membership(id):
    member = find_member(id, {"membership": 1})
    data = validate_body({"plan": {"type": "string", "required": True, "choices": list(PLAN_DAYS)}})
    current = member.get("membership") or {}
    now = now_utc()

    if current.get("end_date") and current["end_date"] >= now:
        # Still active: extend from the current end date so no paid days are lost
        membership = {**current, "plan": data["plan"], "status": "active",
                      "end_date": current["end_date"] + timedelta(days=PLAN_DAYS[data["plan"]])}
        action = "renewed"
    else:
        # Expired (or never had one): a new period starts today
        membership = new_membership(data["plan"])
        action = "restarted"
    members.update_one({"_id": member["_id"]}, {"$set": {"membership": membership}})
    return ok({"message": f"Membership {action}", "membership": membership})


@members_bp.route("/members/<id>/stats", methods=["GET"])
@jwt_required
def member_stats(id):
    member = find_member(id, {"name": 1})
    args = validate_query({"weeks": {"type": "int", "min": 1, "max": 52, "default": 8}})
    since = now_utc() - timedelta(weeks=args["weeks"])

    pipeline = [
        {"$match": {"_id": member["_id"]}},
        {"$unwind": "$workouts"},                       # one document per workout
        {"$replaceRoot": {"newRoot": "$workouts"}},     # work with the workout fields directly
        {"$match": {"date": {"$gte": since}}},
        {"$facet": {                                    # several summaries in one pass
            "totals": [
                {"$group": {"_id": None,
                            "sessions": {"$sum": 1},
                            "total_minutes": {"$sum": "$duration_min"},
                            "total_calories": {"$sum": "$calories_burned"},
                            "avg_duration_min": {"$avg": "$duration_min"},
                            "avg_calories": {"$avg": "$calories_burned"},
                            "avg_bpm": {"$avg": "$avg_bpm"},
                            "peak_bpm": {"$max": "$max_bpm"},
                            "first_workout": {"$min": "$date"},
                            "last_workout": {"$max": "$date"}}},
                {"$project": {"_id": 0, "sessions": 1, "total_minutes": 1, "total_calories": 1,
```

```
                                "peak_bpm": 1, "first_workout": 1, "last_workout": 1,
                                "avg_duration_min": {"$round": ["$avg_duration_min", 1]},
                                "avg_calories": {"$round": ["$avg_calories", 1]},
                                "avg_bpm": {"$round": ["$avg_bpm", 1]}}},
            ],
            "weekly": [
                # $dateTrunc rounds each date down to the Monday of its week
                {"$group": {"_id": {"$dateTrunc": {"date": "$date", "unit": "week",
                                                   "startOfWeek": "monday"}},
                            "sessions": {"$sum": 1},
                            "minutes": {"$sum": "$duration_min"},
                            "calories": {"$sum": "$calories_burned"},
                            "avg_bpm": {"$avg": "$avg_bpm"}}},
                {"$sort": {"_id": 1}},
                {"$project": {"_id": 0, "week_start": "$_id", "sessions": 1, "minutes": 1,
                              "calories": 1, "avg_bpm": {"$round": ["$avg_bpm", 1]}}},
            ],
            "by_type": [
                {"$group": {"_id": "$workout_type", "sessions": {"$sum": 1},
                            "minutes": {"$sum": "$duration_min"},
                            "calories": {"$sum": "$calories_burned"}}},
                {"$sort": {"sessions": -1, "minutes": -1}},
                {"$project": {"_id": 0, "workout_type": "$_id", "sessions": 1, "minutes": 1,
                              "calories": 1}},
            ],
            "top_exercises": [
                {"$unwind": "$exercises"},
                {"$group": {"_id": "$exercises.exercise_id",
                            "title": {"$first": "$exercises.title"},
                            "times_done": {"$sum": 1},
                            "total_sets": {"$sum": "$exercises.sets"},
                            "total_reps": {"$sum": {"$multiply": ["$exercises.sets", "$exercises.rep
  >>   s"]}},
                            "max_weight_kg": {"$max": "$exercises.weight_kg"}}},
                {"$sort": {"times_done": -1, "total_sets": -1}},
                {"$limit": 5},
                {"$project": {"_id": 0, "exercise_id": "$_id", "title": 1, "times_done": 1,
                              "total_sets": 1, "total_reps": 1, "max_weight_kg": 1}},
            ],
        }},
    ]
    result = list(members.aggregate(pipeline))
    stats = result[0] if result else {"totals": [], "weekly": [], "by_type": [], "top_exercises": []
  >>   }
    return ok({
        "member": member,
        "period": {"weeks": args["weeks"], "from": since, "to": now_utc()},
        "totals": stats["totals"][0] if stats["totals"] else {"sessions": 0},
        "favourite_workout_type": stats["by_type"][0]["workout_type"] if stats["by_type"] else None,
        "weekly": stats["weekly"],
        "by_type": stats["by_type"],
        "top_exercises": stats["top_exercises"],
    })


@members_bp.route("/members/<id>/bookings", methods=["GET"])
@jwt_required
def member_bookings(id):
    member = find_member(id, {"_id": 1})
    args = validate_query({
        "status": {"type": "string", "choices": BOOKING_STATUSES},
        "when": {"type": "string", "choices": ["upcoming", "past", "all"], "default": "all"},
        **PAGINATION_QUERY,
    })
    mid = member["_id"]
    class_match = {"bookings.member_id": mid}          # uses the bookings.member_id index
    if args["when"] == "upcoming":
        class_match["start_time"] = {"$gt": now_utc()}
    elif args["when"] == "past":
        class_match["start_time"] = {"$lte": now_utc()}
    booking_match = {"bookings.member_id": mid}
    if "status" in args:
        booking_match["bookings.status"] = args["status"]

    pipeline = [
        {"$match": class_match},
        {"$unwind": "$bookings"},
        {"$match": booking_match},                     # keep only this member's bookings
        {"$sort": {"start_time": 1 if args["when"] == "upcoming" else -1}},
        {"$project": {
```

```
            "_id": "$bookings._id",
            "status": "$bookings.status",
            "booked_at": "$bookings.booked_at",
            "attended": "$bookings.attended",
            "class": {"_id": "$_id", "title": "$title", "category": "$category",
                      "branch_id": "$branch_id", "branch_name": "$branch_name",
                      "trainer_name": "$trainer_name", "start_time": "$start_time",
                      "duration_min": "$duration_min", "status": "$status"},
        }},
        facet_page(args["pn"], args["ps"]),
    ]
    return ok(facet_envelope(list(classes.aggregate(pipeline)), args["pn"], args["ps"]))
```

blueprints/workouts/workouts.py

```
"""
workouts Blueprint - workout logs embedded in each member document (sub-documents).

    GET    /api/v1.0/members/<id>/workouts                list (date range, type, paging)
    POST   /api/v1.0/members/<id>/workouts                log a workout
    GET    /api/v1.0/members/<id>/workouts/<workout_id>   one workout
    PUT    /api/v1.0/members/<id>/workouts/<workout_id>   update some fields
    DELETE /api/v1.0/members/<id>/workouts/<workout_id>   delete
All routes: the member themself or an admin.
"""

from bson import ObjectId
from flask import Blueprint

from blueprints.members.members import find_member
from decorators import jwt_required
from globals import exercises, members
from utils.constants import WORKOUT_TYPES
from utils.errors import ApiError
from utils.pagination import PAGINATION_QUERY, facet_envelope, facet_page
from utils.responses import created, no_content, ok
from utils.validation import now_utc, parse_object_id, validate_body, validate_query

workouts_bp = Blueprint("workouts", __name__)

EXERCISE_ENTRY_SCHEMA = {
    "exercise_id": {"type": "objectid", "required": True},
    "sets": {"type": "int", "required": True, "min": 1, "max": 20},
    "reps": {"type": "int", "required": True, "min": 1, "max": 100},
    "weight_kg": {"type": "number", "nullable": True, "min": 0, "max": 500},
}

WORKOUT_SCHEMA = {
    "date": {"type": "datetime", "required": True},
    "workout_type": {"type": "string", "required": True, "choices": WORKOUT_TYPES},
    "duration_min": {"type": "int", "required": True, "min": 1, "max": 300},
    "calories_burned": {"type": "int", "required": True, "min": 0, "max": 5000},
    "avg_bpm": {"type": "int", "required": True, "min": 40, "max": 220},
    "max_bpm": {"type": "int", "required": True, "min": 40, "max": 230},
    "exercises": {"type": "list", "max_items": 30,
                  "items": {"type": "object", "schema": EXERCISE_ENTRY_SCHEMA}},
}


def _check_rules(workout):
    """Checks that involve more than one field or the current time."""
    errors = {}
    if workout.get("date") and workout["date"] > now_utc():
        errors["date"] = "A workout cannot be logged in the future"
    if workout.get("avg_bpm") and workout.get("max_bpm") and workout["max_bpm"] < workout["avg_bpm"]
 >>   :
        errors["max_bpm"] = "Must be greater than or equal to avg_bpm"
    if errors:
        raise ApiError(400, "Validation failed", errors)


def _resolve_exercises(entries):
    """Looks up every exercise_id in one query and copies in its title."""
    ids = [e["exercise_id"] for e in entries]
    titles = {e["_id"]: e["title"] for e in exercises.find({"_id": {"$in": ids}}, {"title": 1})}
    missing = {f"exercises[{i}].exercise_id": "No exercise exists with this id"
               for i, e in enumerate(entries) if e["exercise_id"] not in titles}
    if missing:
        raise ApiError(400, "Validation failed", missing)
    return [{"exercise_id": e["exercise_id"], "title": titles[e["exercise_id"]],
             "sets": e["sets"], "reps": e["reps"], "weight_kg": e.get("weight_kg")}
            for e in entries]


def _find_workout(member_id, workout_id):
    member = find_member(member_id, {"_id": 1})
    wid = parse_object_id(workout_id, "workout id")
    doc = members.find_one({"_id": member["_id"], "workouts._id": wid}, {"workouts.$": 1})
    if not doc:
        raise ApiError(404, "Workout not found")
    return member["_id"], doc["workouts"][0]


```

```
@workouts_bp.route("/members/<id>/workouts", methods=["GET"])
@jwt_required
def list_workouts(id):
    member = find_member(id, {"_id": 1})
    args = validate_query({
        "from": {"type": "datetime"},
        "to": {"type": "datetime"},
        "type": {"type": "list", "items": {"type": "string", "choices": WORKOUT_TYPES}},
        "sort": {"type": "string", "choices": ["newest", "oldest"], "default": "newest"},
        **PAGINATION_QUERY,
    })
    if "from" in args and "to" in args and args["from"] > args["to"]:
        raise ApiError(400, "Invalid query parameters", {"from": "Must be before 'to'"})

    workout_match = {}
    if "from" in args or "to" in args:
        workout_match["date"] = {}
        if "from" in args:
            workout_match["date"]["$gte"] = args["from"]
        if "to" in args:
            workout_match["date"]["$lte"] = args["to"]
    if "type" in args:
        workout_match["workout_type"] = {"$in": args["type"]}

    pipeline = [
        {"$match": {"_id": member["_id"]}},
        {"$unwind": "$workouts"},
        {"$replaceRoot": {"newRoot": "$workouts"}},
        {"$match": workout_match},
        {"$sort": {"date": -1 if args["sort"] == "newest" else 1}},
        facet_page(args["pn"], args["ps"]),
    ]
    return ok(facet_envelope(list(members.aggregate(pipeline)), args["pn"], args["ps"]))


@workouts_bp.route("/members/<id>/workouts", methods=["POST"])
@jwt_required
def add_workout(id):
    member = find_member(id, {"_id": 1})
    data = validate_body(WORKOUT_SCHEMA)
    _check_rules(data)
    workout = {"_id": ObjectId(), **data,
               "exercises": _resolve_exercises(data.get("exercises", []))}
    members.update_one({"_id": member["_id"]}, {"$push": {"workouts": workout}})
    return created("Workout logged", workout["_id"], "workouts.get_workout",
                   id=member["_id"], workout_id=workout["_id"])


@workouts_bp.route("/members/<id>/workouts/<workout_id>", methods=["GET"])
@jwt_required
def get_workout(id, workout_id):
    return ok(_find_workout(id, workout_id)[1])


@workouts_bp.route("/members/<id>/workouts/<workout_id>", methods=["PUT"])
@jwt_required
def update_workout(id, workout_id):
    member_id, workout = _find_workout(id, workout_id)
    data = validate_body(WORKOUT_SCHEMA, partial=True)
    _check_rules({**workout, **data})              # check against the merged result
    if "exercises" in data:
        data["exercises"] = _resolve_exercises(data["exercises"])
    updates = {f"workouts.$.{field}": value for field, value in data.items()}
    members.update_one({"_id": member_id, "workouts._id": workout["_id"]}, {"$set": updates})
    return ok({"message": "Workout updated", "workout": _find_workout(id, workout_id)[1]})


@workouts_bp.route("/members/<id>/workouts/<workout_id>", methods=["DELETE"])
@jwt_required
def delete_workout(id, workout_id):
    member_id, workout = _find_workout(id, workout_id)
    members.update_one({"_id": member_id}, {"$pull": {"workouts": {"_id": workout["_id"]}}})
    return no_content()
```

blueprints/classes/classes.py

```
"""
classes Blueprint - the class timetable.

    GET    /api/v1.0/classes          list with filters, sorted by start_time
    POST   /api/v1.0/classes          create                    (admin, trainer)
    GET    /api/v1.0/classes/<id>     one class (without the bookings list)
    PUT    /api/v1.0/classes/<id>     update / cancel           (admin, the class's trainer)
    DELETE /api/v1.0/classes/<id>     delete if no active bookings (admin, the class's trainer)
"""

import re
from datetime import timedelta

from flask import Blueprint, g

from decorators import is_admin, role_required
from globals import branches, classes, users
from utils.constants import CLASS_LEVELS, CLASS_STATUSES, WORKOUT_TYPES
from utils.errors import ApiError
from utils.pagination import PAGINATION_QUERY, paginate_find
from utils.responses import created, no_content, ok
from utils.validation import now_utc, parse_object_id, validate_body, validate_query

classes_bp = Blueprint("classes", __name__)

CLASS_SCHEMA = {
    "title": {"type": "string", "required": True, "min_len": 3, "max_len": 80},
    "category": {"type": "string", "required": True, "choices": WORKOUT_TYPES},
    "description": {"type": "string", "max_len": 500},
    "branch_id": {"type": "objectid", "required": True},
    "trainer_id": {"type": "objectid"},            # trainers default to themselves
    "start_time": {"type": "datetime", "required": True},
    "duration_min": {"type": "int", "required": True, "min": 1, "max": 300},
    "level": {"type": "string", "required": True, "choices": CLASS_LEVELS},
    "capacity": {"type": "int", "required": True, "min": 1, "max": 100},
}
# A class cannot move branch (create a new one instead); status can be changed on update
UPDATE_SCHEMA = {**{k: v for k, v in CLASS_SCHEMA.items() if k != "branch_id"},
                 "status": {"type": "string", "choices": CLASS_STATUSES}}

NO_BOOKINGS = {"bookings": 0}


def find_class(class_id, projection=None):
    """Shared with the bookings Blueprint."""
    cls = classes.find_one({"_id": parse_object_id(class_id, "class id")}, projection)
    if not cls:
        raise ApiError(404, "Class not found")
    return cls


def ensure_can_manage(cls):
    """Admins manage every class; trainers only the classes they teach."""
    if is_admin():
        return
    if g.user["role"] == "trainer" and str(cls["trainer_id"]) == g.user["user_id"]:
        return
    raise ApiError(403, "Only an admin or the class's own trainer can do this")


def _get_trainer(trainer_id):
    trainer = users.find_one({"_id": trainer_id, "role": "trainer"}, {"name": 1})
    if not trainer:
        raise ApiError(400, "Validation failed", {"trainer_id": "No trainer exists with this id"})
    return trainer


def _check_trainer_free(trainer_id, start, duration_min, exclude_id=None):
    """409 if the trainer already teaches a scheduled class that overlaps this time.

    Two time ranges overlap when each one starts before the other ends:
        existing.start < new.end  AND  existing.end > new.start
    existing.end is not stored, so $expr works it out as start_time + duration.
    """
    end = start + timedelta(minutes=duration_min)
    query = {
        "trainer_id": trainer_id,
        "status": "scheduled",
```

```
        "start_time": {"$lt": end},
        "$expr": {"$gt": [{"$add": ["$start_time", {"$multiply": ["$duration_min", 60000]}]}, start]
    },
    }
    if exclude_id:
        query["_id"] = {"$ne": exclude_id}
    clash = classes.find_one(query, {"title": 1, "start_time": 1, "branch_name": 1})
    if clash:
        raise ApiError(409, "The trainer already has a class at this time",
                       {"clashing_class": clash})


def _add_spaces(cls):
    cls["spaces_left"] = max(cls["capacity"] - cls["booked_count"], 0)
    return cls


@classes_bp.route("/classes", methods=["GET"])
def list_classes():
    args = validate_query({
        "branch": {"type": "objectid"},
        "trainer": {"type": "objectid"},
        "category": {"type": "list", "items": {"type": "string", "choices": WORKOUT_TYPES}},
        "level": {"type": "list", "items": {"type": "string", "choices": CLASS_LEVELS}},
        "status": {"type": "string", "choices": CLASS_STATUSES},
        "when": {"type": "string", "choices": ["upcoming", "past", "all"], "default": "upcoming"},
        "from": {"type": "datetime"},
        "to": {"type": "datetime"},
        "available": {"type": "bool"},
        "q": {"type": "string", "max_len": 80},
        **PAGINATION_QUERY,
    })
    if "from" in args and "to" in args and args["from"] > args["to"]:
        raise ApiError(400, "Invalid query parameters", {"from": "Must be before 'to'"})

    now = now_utc()
    time_range = {}
    if args["when"] == "upcoming":
        time_range["$gt"] = now
    elif args["when"] == "past":
        time_range["$lte"] = now
    if "from" in args:
        time_range["$gte"] = args["from"]
    if "to" in args:
        time_range["$lte"] = min(args["to"], time_range.get("$lte", args["to"]))

    query = {}
    if time_range:
        query["start_time"] = time_range
    if "branch" in args:
        query["branch_id"] = args["branch"]
    if "trainer" in args:
        query["trainer_id"] = args["trainer"]
    if "category" in args:
        query["category"] = {"$in": args["category"]}
    if "level" in args:
        query["level"] = {"$in": args["level"]}
    if "status" in args:
        query["status"] = args["status"]
    if "q" in args:
        query["title"] = {"$regex": re.escape(args["q"]), "$options": "i"}
    if args.get("available"):
        # Bookable = scheduled, in the future, and booked_count < capacity.
        # $expr is needed to compare two fields of the same document.
        query["status"] = "scheduled"
        query.setdefault("start_time", {})["$gt"] = now
        query["$expr"] = {"$lt": ["$booked_count", "$capacity"]}

    page = paginate_find(classes, query, args["pn"], args["ps"], projection=NO_BOOKINGS,
                         sort=[("start_time", 1), ("_id", 1)])
    page["data"] = [_add_spaces(c) for c in page["data"]]
    return ok(page)


@classes_bp.route("/classes", methods=["POST"])
@role_required("admin", "trainer")
def create_class():
    data = validate_body(CLASS_SCHEMA)

    if g.user["role"] == "trainer":
```

```
        own_id = parse_object_id(g.user["user_id"])
        if data.get("trainer_id", own_id) != own_id:
            raise ApiError(403, "Trainers can only create classes they teach themselves")
        data["trainer_id"] = own_id
    elif "trainer_id" not in data:
        raise ApiError(400, "Validation failed", {"trainer_id": "This field is required"})

    errors = {}
    if data["start_time"] <= now_utc():
        errors["start_time"] = "Must be in the future"
    branch = branches.find_one({"_id": data["branch_id"]}, {"name": 1})
    if not branch:
        errors["branch_id"] = "No branch exists with this id"
    if errors:
        raise ApiError(400, "Validation failed", errors)
    trainer = _get_trainer(data["trainer_id"])
    _check_trainer_free(data["trainer_id"], data["start_time"], data["duration_min"])

    doc = {
        "title": data["title"],
        "category": data["category"],
        "description": data.get("description") or
        f"{data['title']} is a {data['duration_min']}-minute {data['category'].lower()} class "
        f"for {data['level'].lower()} members.",
        "branch_id": branch["_id"],
        "branch_name": branch["name"],          # copied so listings need no lookup
        "trainer_id": trainer["_id"],
        "trainer_name": trainer["name"],
        "start_time": data["start_time"],
        "duration_min": data["duration_min"],
        "level": data["level"],
        "capacity": data["capacity"],
        "booked_count": 0,
        "status": "scheduled",
        "bookings": [],
    }
    new_id = classes.insert_one(doc).inserted_id
    return created("Class created", new_id, "classes.get_class", id=new_id)


@classes_bp.route("/classes/<id>", methods=["GET"])
def get_class(id):
    return ok(_add_spaces(find_class(id, NO_BOOKINGS)))


@classes_bp.route("/classes/<id>", methods=["PUT"])
@role_required("admin", "trainer")
def update_class(id):
    cls = find_class(id, NO_BOOKINGS)
    ensure_can_manage(cls)
    data = validate_body(UPDATE_SCHEMA, partial=True)
    now = now_utc()

    if cls["status"] == "cancelled":
        raise ApiError(400, "A cancelled class cannot be changed")
    if "trainer_id" in data and not is_admin() and data["trainer_id"] != cls["trainer_id"]:
        raise ApiError(403, "Only an admin can reassign a class to another trainer")
    if "start_time" in data and data["start_time"] <= now:
        raise ApiError(400, "Validation failed", {"start_time": "Must be in the future"})
    if "capacity" in data and data["capacity"] < cls["booked_count"]:
        raise ApiError(409, "Capacity cannot be lower than the number of places already booked",
                       {"booked_count": cls["booked_count"]})
    if data.get("status") == "completed" and cls["start_time"] > now:
        raise ApiError(400, "A class cannot be completed before it has started")
    if data.get("status") == "scheduled" and cls["status"] == "completed":
        raise ApiError(400, "A completed class cannot be rescheduled")

    updates = dict(data)
    if "trainer_id" in data:
        updates["trainer_name"] = _get_trainer(data["trainer_id"])["name"]
    if {"trainer_id", "start_time", "duration_min"} & data.keys():
        _check_trainer_free(data.get("trainer_id", cls["trainer_id"]),
                            data.get("start_time", cls["start_time"]),
                            data.get("duration_min", cls["duration_min"]), exclude_id=cls["_id"])

    if data.get("status") == "cancelled":
        # Cancel every active booking in the same update that cancels the class
        updates["booked_count"] = 0
        updates["bookings.$[b].status"] = "cancelled"
        classes.update_one({"_id": cls["_id"]}, {"$set": updates},
```

```
                            array_filters=[{"b.status": "booked"}])
        message = f"Class cancelled; {cls['booked_count']} booking(s) cancelled"
    else:
        classes.update_one({"_id": cls["_id"]}, {"$set": updates})
        message = "Class updated"
    return ok({"message": message, "class": _add_spaces(find_class(id, NO_BOOKINGS))})


@classes_bp.route("/classes/<id>", methods=["DELETE"])
@role_required("admin", "trainer")
def delete_class(id):
    cls = find_class(id, NO_BOOKINGS)
    ensure_can_manage(cls)
    if cls["status"] == "scheduled" and cls["booked_count"] > 0:
        raise ApiError(409, "This class has active bookings",
                       {"booked_count": cls["booked_count"],
                        "hint": "Cancel it first with PUT {\"status\": \"cancelled\"}"})
    classes.delete_one({"_id": cls["_id"]})
    return no_content()
```

blueprints/bookings/bookings.py

```
"""
bookings Blueprint - class bookings embedded in each class document (sub-documents).

    POST   /api/v1.0/classes/<id>/bookings                          book a place       (member)
    GET    /api/v1.0/classes/<id>/bookings                          list               (admin, clas
>>  s trainer)
    GET    /api/v1.0/classes/<id>/bookings/<booking_id>             one booking        (admin, clas
>>  s trainer, booker)
    DELETE /api/v1.0/classes/<id>/bookings/<booking_id>             cancel             (admin, clas
>>  s trainer, booker)
    PUT    /api/v1.0/classes/<id>/bookings/<booking_id>/attendance  mark attendance    (admin, clas
>>  s trainer)
"""

from bson import ObjectId
from flask import Blueprint, g

from blueprints.classes.classes import ensure_can_manage, find_class
from decorators import current_member_id, jwt_required, role_required
from globals import classes, members
from utils.constants import BOOKING_STATUSES
from utils.errors import ApiError
from utils.responses import created, ok
from utils.validation import now_utc, parse_object_id, validate_body, validate_query

bookings_bp = Blueprint("bookings", __name__)


def _find_booking(class_id, booking_id):
    """Returns (class without bookings, booking)."""
    cls = find_class(class_id, {"bookings": 0})
    bid = parse_object_id(booking_id, "booking id")
    doc = classes.find_one({"_id": cls["_id"], "bookings._id": bid}, {"bookings.$": 1})
    if not doc:
        raise ApiError(404, "Booking not found")
    return cls, doc["bookings"][0]


def _ensure_booking_access(cls, booking):
    """The member who made the booking, the class's trainer or an admin."""
    if g.user.get("member_id") == str(booking["member_id"]):
        return
    ensure_can_manage(cls)


@bookings_bp.route("/classes/<id>/bookings", methods=["POST"])
@jwt_required
def book_class(id):
    member_id = current_member_id()                 # only member accounts can book
    cls = find_class(id, {"start_time": 1})
    member = members.find_one({"_id": member_id}, {"name": 1, "membership": 1})
    if not member:
        raise ApiError(404, "Your member record no longer exists")

    end_date = (member.get("membership") or {}).get("end_date")
    if not end_date or end_date < now_utc():
        raise ApiError(403, "Your membership is not active", {"membership_end_date": end_date})
    if end_date < cls["start_time"]:
        raise ApiError(403, "Your membership ends before this class starts",
                       {"membership_end_date": end_date, "class_start_time": cls["start_time"]})

    booking = {"_id": ObjectId(), "member_id": member_id, "member_name": member["name"],
               "booked_at": now_utc(), "status": "booked", "attended": None}

    # ATOMIC BOOKING: every rule is part of the filter of a single update_one.
    # MongoDB checks the filter and applies the update to the document as one
    # operation, so two members can never both take the last place.
    result = classes.update_one(
        {
            "_id": cls["_id"],
            "status": "scheduled",
            "start_time": {"$gt": now_utc()},
            "$expr": {"$lt": ["$booked_count", "$capacity"]},          # a place is free
            "bookings": {"$not": {"$elemMatch": {"member_id": member_id, "status": "booked"}}},
        },
        {"$push": {"bookings": booking}, "$inc": {"booked_count": 1}},
    )

```

```
    if result.modified_count == 0:
        # Nothing changed, so one of the rules failed - read the class to say which one
        current = classes.find_one({"_id": cls["_id"]}, {
            "status": 1, "start_time": 1, "capacity": 1, "booked_count": 1,
            "bookings": {"$elemMatch": {"member_id": member_id, "status": "booked"}}})
        if current["status"] == "cancelled":
            raise ApiError(400, "This class has been cancelled")
        if current["status"] == "completed" or current["start_time"] <= now_utc():
            raise ApiError(400, "This class has already started or taken place")
        if current.get("bookings"):
            raise ApiError(409, "You have already booked this class",
                           {"booking_id": current["bookings"][0]["_id"]})
        if current["booked_count"] >= current["capacity"]:
            raise ApiError(409, "This class is full", {"capacity": current["capacity"]})
        raise ApiError(409, "The class changed while booking, please try again")

    updated = classes.find_one({"_id": cls["_id"]}, {"capacity": 1, "booked_count": 1})
    return created("Class booked", booking["_id"], "bookings.get_booking",
                   extra={"spaces_left": updated["capacity"] - updated["booked_count"]},
                   id=cls["_id"], booking_id=booking["_id"])


@bookings_bp.route("/classes/<id>/bookings", methods=["GET"])
@role_required("admin", "trainer")
def list_bookings(id):
    cls = find_class(id)
    ensure_can_manage(cls)
    args = validate_query({"status": {"type": "string", "choices": BOOKING_STATUSES}})
    bookings = cls.pop("bookings")
    data = [b for b in bookings if "status" not in args or b["status"] == args["status"]]
    data.sort(key=lambda b: b["booked_at"])
    return ok({
        "class": {"_id": cls["_id"], "title": cls["title"], "start_time": cls["start_time"],
                  "capacity": cls["capacity"], "booked_count": cls["booked_count"],
                  "status": cls["status"]},
        "data": data,
        "count": len(data),
        "summary": {
            "booked": sum(b["status"] == "booked" for b in bookings),
            "cancelled": sum(b["status"] == "cancelled" for b in bookings),
            "attended": sum(b["attended"] is True for b in bookings),
            "no_shows": sum(b["attended"] is False for b in bookings),
        },
    })


@bookings_bp.route("/classes/<id>/bookings/<booking_id>", methods=["GET"])
@jwt_required
def get_booking(id, booking_id):
    cls, booking = _find_booking(id, booking_id)
    _ensure_booking_access(cls, booking)
    return ok({**booking, "class": {"_id": cls["_id"], "title": cls["title"],
                                    "start_time": cls["start_time"], "branch_name": cls["branch_name
>>   "],
                                    "status": cls["status"]}})


@bookings_bp.route("/classes/<id>/bookings/<booking_id>", methods=["DELETE"])
@jwt_required
def cancel_booking(id, booking_id):
    cls, booking = _find_booking(id, booking_id)
    _ensure_booking_access(cls, booking)
    if booking["status"] == "cancelled":
        raise ApiError(409, "This booking is already cancelled")

    # Same idea as booking: the rules are in the filter, and the status change and the
    # booked_count decrement happen in one atomic update. "bookings.$" is the element
    # matched by the $elemMatch.
    result = classes.update_one(
        {"_id": cls["_id"], "status": "scheduled", "start_time": {"$gt": now_utc()},
         "bookings": {"$elemMatch": {"_id": booking["_id"], "status": "booked"}}},
        {"$set": {"bookings.$.status": "cancelled"}, "$inc": {"booked_count": -1}},
    )
    if result.modified_count == 0:
        raise ApiError(400, "Bookings can only be cancelled before a scheduled class starts")
    return ok({"message": "Booking cancelled", "booking_id": booking["_id"]})


@bookings_bp.route("/classes/<id>/bookings/<booking_id>/attendance", methods=["PUT"])
@role_required("admin", "trainer")
```

```
def mark_attendance(id, booking_id):
    cls, booking = _find_booking(id, booking_id)
    ensure_can_manage(cls)
    data = validate_body({"attended": {"type": "bool", "required": True}})

    if cls["status"] == "cancelled":
        raise ApiError(400, "This class was cancelled")
    if cls["start_time"] > now_utc():
        raise ApiError(400, "Attendance can only be recorded once the class has started",
                       {"start_time": cls["start_time"]})
    if booking["status"] != "booked":
        raise ApiError(400, "Attendance cannot be recorded for a cancelled booking")

    updates = {"bookings.$.attended": data["attended"]}
    if cls["status"] == "scheduled":
        updates["status"] = "completed"       # recording attendance means the class took place
    classes.update_one({"_id": cls["_id"], "bookings._id": booking["_id"]}, {"$set": updates})
    return ok({"message": "Attendance recorded", "booking": _find_booking(id, booking_id)[1]})
```

blueprints/reports/reports.py

```
"""
reports Blueprint - management reports built with aggregation pipelines (admin only).

    GET /api/v1.0/reports/popular-classes        bookings, fill rate, attendance by class
    GET /api/v1.0/reports/branch-usage           members, bookings and fill rate per branch
    GET /api/v1.0/reports/peak-times             bookings by weekday and hour
    GET /api/v1.0/reports/trainer-performance    classes taught, fill and attendance per trainer
    GET /api/v1.0/reports/membership-summary     plans, statuses and memberships expiring soon

Definitions used throughout:
    fill rate        = booked places / capacity
    attendance rate  = bookings marked attended / active bookings, for classes that have
                       already happened (start_time in the past)
"""

from datetime import timedelta

from flask import Blueprint

from decorators import role_required
from globals import branches, classes, members
from utils.constants import WORKOUT_TYPES
from utils.errors import ApiError
from utils.responses import ok
from utils.validation import now_utc, validate_query

reports_bp = Blueprint("reports", __name__)

DATE_RANGE_QUERY = {"from": {"type": "datetime"}, "to": {"type": "datetime"},
                    "branch": {"type": "objectid"}}

# Expression: number of bookings in the class's bookings array marked attended
ATTENDED_COUNT = {"$size": {"$filter": {"input": "$bookings",
                                        "cond": {"$eq": ["$$this.attended", True]}}}}


def _percent(numerator, denominator):
    """Expression: numerator / denominator as a percentage (1 d.p.), or null if dividing by 0."""
    return {"$cond": [{"$gt": [denominator, 0]},
                      {"$round": [{"$multiply": [{"$divide": [numerator, denominator]}, 100]}, 1]},
                      None]}


def _class_match(args):
    """$match for classes that were not cancelled, within the optional date range / branch."""
    if "from" in args and "to" in args and args["from"] > args["to"]:
        raise ApiError(400, "Invalid query parameters", {"from": "Must be before 'to'"})
    match = {"status": {"$ne": "cancelled"}}
    if "from" in args or "to" in args:
        match["start_time"] = {}
        if "from" in args:
            match["start_time"]["$gte"] = args["from"]
        if "to" in args:
            match["start_time"]["$lte"] = args["to"]
    if "branch" in args:
        match["branch_id"] = args["branch"]
    return match


def _class_fields(now):
    """$project stage shared by the class-based reports."""
    happened = {"$lte": ["$start_time", now]}
    return {"$project": {
        "title": 1, "category": 1, "trainer_id": 1, "trainer_name": 1, "branch_id": 1,
        "start_time": 1, "capacity": 1, "booked_count": 1,
        "happened": happened,
        "attended": ATTENDED_COUNT,
        # only bookings for classes that have happened count towards attendance
        "past_booked": {"$cond": [happened, "$booked_count", 0]},
    }}


@reports_bp.route("/reports/popular-classes", methods=["GET"])
@role_required("admin")
def popular_classes():
    args = validate_query({
        **DATE_RANGE_QUERY,
        "category": {"type": "list", "items": {"type": "string", "choices": WORKOUT_TYPES}},
```

```
        "sort": {"type": "string", "choices": ["bookings", "fill_rate", "attendance_rate"],
                 "default": "bookings"},
        "limit": {"type": "int", "min": 1, "max": 50, "default": 10},
    })
    match = _class_match(args)
    if "category" in args:
        match["category"] = {"$in": args["category"]}

    def summary(group_id):
        """Group stage + rate calculation, reused for per-class and per-category results."""
        return [
            {"$group": {"_id": group_id,
                        "classes": {"$sum": 1},
                        "bookings": {"$sum": "$booked_count"},
                        "capacity": {"$sum": "$capacity"},
                        "attended": {"$sum": "$attended"},
                        "past_booked": {"$sum": "$past_booked"}}},
            {"$addFields": {"fill_rate": _percent("$bookings", "$capacity"),
                            "attendance_rate": _percent("$attended", "$past_booked")}},
        ]

    pipeline = [
        {"$match": match},
        _class_fields(now_utc()),
        {"$facet": {
            "by_class": summary({"category": "$category", "title": "$title"}) + [
                {"$sort": {args["sort"]: -1, "bookings": -1}},
                {"$limit": args["limit"]},
                {"$project": {"_id": 0, "category": "$_id.category", "title": "$_id.title",
                              "classes": 1, "bookings": 1, "capacity": 1, "fill_rate": 1,
                              "attended": 1, "attendance_rate": 1}},
            ],
            "by_category": summary("$category") + [
                {"$sort": {"bookings": -1}},
                {"$project": {"_id": 0, "category": "$_id", "classes": 1, "bookings": 1,
                              "fill_rate": 1, "attendance_rate": 1}},
            ],
        }},
    ]
    result = list(classes.aggregate(pipeline))[0]
    return ok({"sorted_by": args["sort"], **result})


@reports_bp.route("/reports/branch-usage", methods=["GET"])
@role_required("admin")
def branch_usage():
    args = validate_query({"from": DATE_RANGE_QUERY["from"], "to": DATE_RANGE_QUERY["to"]})
    class_match = _class_match(args)
    now = now_utc()

    pipeline = [
        # $lookup with a sub-pipeline: count this branch's members without
        # copying every member document into the result
        {"$lookup": {
            "from": "members",
            "let": {"branch_id": "$_id"},
            "pipeline": [
                {"$match": {"$expr": {"$eq": ["$home_branch_id", "$$branch_id"]}}},
                {"$group": {"_id": None,
                            "total": {"$sum": 1},
                            "active": {"$sum": {"$cond": [{"$gte": ["$membership.end_date", now]}, 1
 >>   , 0]}}}},
            ],
            "as": "member_stats",
        }},
        {"$lookup": {
            "from": "classes",
            "let": {"branch_id": "$_id"},
            "pipeline": [
                {"$match": {**class_match, "$expr": {"$eq": ["$branch_id", "$$branch_id"]}}},
                _class_fields(now),
                {"$group": {"_id": None,
                            "classes": {"$sum": 1},
                            "bookings": {"$sum": "$booked_count"},
                            "capacity": {"$sum": "$capacity"},
                            "attended": {"$sum": "$attended"},
                            "past_booked": {"$sum": "$past_booked"}}},
            ],
            "as": "class_stats",
        }},
```

```
        # Each lookup returns a one-element array (or empty): take the first element
        {"$set": {"m": {"$ifNull": [{"$first": "$member_stats"}, {}]},
                  "c": {"$ifNull": [{"$first": "$class_stats"}, {}]}}},
        {"$project": {
            "_id": 0, "branch_id": "$_id", "name": 1, "city": "$address.city",
            "members": {"$ifNull": ["$m.total", 0]},
            "active_members": {"$ifNull": ["$m.active", 0]},
            "classes": {"$ifNull": ["$c.classes", 0]},
            "bookings": {"$ifNull": ["$c.bookings", 0]},
            "fill_rate": _percent({"$ifNull": ["$c.bookings", 0]}, {"$ifNull": ["$c.capacity", 0]}),
            "attendance_rate": _percent({"$ifNull": ["$c.attended", 0]}, {"$ifNull": ["$c.past_booke
  >>   d", 0]}),
            "bookings_per_active_member": {"$cond": [
                {"$gt": ["$m.active", 0]},
                {"$round": [{"$divide": ["$c.bookings", "$m.active"]}, 2]}, None]},
        }},
        {"$sort": {"bookings": -1}},
    ]
    data = list(branches.aggregate(pipeline))
    return ok({"data": data, "count": len(data)})


@reports_bp.route("/reports/peak-times", methods=["GET"])
@role_required("admin")
def peak_times():
    args = validate_query({**DATE_RANGE_QUERY,
                           "limit": {"type": "int", "min": 1, "max": 50, "default": 10}})
    day_names = ["", "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]

    pipeline = [
        {"$match": _class_match(args)},
        {"$unwind": "$bookings"},                          # one document per booking
        {"$match": {"bookings.status": "booked"}},
        {"$project": {"class_id": "$_id",
                      "day": {"$dayOfWeek": "$start_time"},     # 1 = Sunday ... 7 = Saturday
                      "hour": {"$hour": "$start_time"}}},
        {"$facet": {
            "busiest_slots": [
                {"$group": {"_id": {"day": "$day", "hour": "$hour"},
                            "bookings": {"$sum": 1}, "classes": {"$addToSet": "$class_id"}}},
                {"$project": {"_id": 0, "day_number": "$_id.day", "hour": "$_id.hour", "bookings": 1
  >>   ,
                              "day": {"$arrayElemAt": [day_names, "$_id.day"]},
                              "classes": {"$size": "$classes"},
                              "avg_bookings_per_class": {"$round": [
                                  {"$divide": ["$bookings", {"$size": "$classes"}]}, 1]}}},
                {"$sort": {"bookings": -1}},
                {"$limit": args["limit"]},
            ],
            "by_weekday": [
                {"$group": {"_id": "$day", "bookings": {"$sum": 1}}},
                {"$sort": {"_id": 1}},
                {"$project": {"_id": 0, "day_number": "$_id", "bookings": 1,
                              "day": {"$arrayElemAt": [day_names, "$_id"]}}},
            ],
            "by_hour": [
                {"$group": {"_id": "$hour", "bookings": {"$sum": 1}}},
                {"$sort": {"_id": 1}},
                {"$project": {"_id": 0, "hour": "$_id", "bookings": 1}},
            ],
        }},
    ]
    result = list(classes.aggregate(pipeline))[0]
    return ok({"timezone": "UTC", **result})


@reports_bp.route("/reports/trainer-performance", methods=["GET"])
@role_required("admin")
def trainer_performance():
    args = validate_query({
        **DATE_RANGE_QUERY,
        "sort": {"type": "string", "choices": ["attendance_rate", "fill_rate", "classes_taught"],
                 "default": "attendance_rate"},
    })
    pipeline = [
        {"$match": _class_match(args)},
        _class_fields(now_utc()),
        {"$group": {"_id": "$trainer_id",
                    "trainer_name": {"$first": "$trainer_name"},
                    "classes_taught": {"$sum": {"$cond": ["$happened", 1, 0]}},
```

```
                    "upcoming_classes": {"$sum": {"$cond": ["$happened", 0, 1]}},
                    "bookings": {"$sum": "$booked_count"},
                    "capacity": {"$sum": "$capacity"},
                    "attended": {"$sum": "$attended"},
                    "past_booked": {"$sum": "$past_booked"},
                    "categories": {"$addToSet": "$category"}}},
        # Join the trainer's account for their specialities and home branch
        {"$lookup": {"from": "users", "localField": "_id", "foreignField": "_id",
                     "as": "trainer", "pipeline": [{"$project": {"specialities": 1, "branch_id": 1}}
 >>   ]}},
        {"$lookup": {"from": "branches", "localField": "trainer.branch_id", "foreignField": "_id",
                     "as": "branch", "pipeline": [{"$project": {"name": 1}}]}},
        {"$project": {
            "_id": 0, "trainer_id": "$_id", "trainer_name": 1,
            "branch": {"$first": "$branch.name"},
            "specialities": {"$first": "$trainer.specialities"},
            "categories": 1, "classes_taught": 1, "upcoming_classes": 1, "bookings": 1,
            "fill_rate": _percent("$bookings", "$capacity"),
            "attendance_rate": _percent("$attended", "$past_booked"),
        }},
        {"$sort": {args["sort"]: -1, "trainer_name": 1}},
    ]
    data = list(classes.aggregate(pipeline))
    return ok({"sorted_by": args["sort"], "data": data, "count": len(data)})


@reports_bp.route("/reports/membership-summary", methods=["GET"])
@role_required("admin")
def membership_summary():
    args = validate_query({"days": {"type": "int", "min": 1, "max": 365, "default": 30},
                           "limit": {"type": "int", "min": 1, "max": 100, "default": 20},
                           "branch": {"type": "objectid"}})
    now = now_utc()
    horizon = now + timedelta(days=args["days"])
    match = {"home_branch_id": args["branch"]} if "branch" in args else {}
    expiring = {"membership.end_date": {"$gte": now, "$lte": horizon}}

    # Status is worked out from end_date, so the report is right even for stale documents
    status_expr = {"$cond": [{"$gte": ["$membership.end_date", now]}, "active", "expired"]}
    pipeline = [
        {"$match": match},
        {"$facet": {
            "total": [{"$count": "members"}],
            "by_status": [
                {"$group": {"_id": status_expr, "members": {"$sum": 1}}},
                {"$project": {"_id": 0, "status": "$_id", "members": 1}},
                {"$sort": {"status": 1}},
            ],
            "by_plan": [
                {"$group": {"_id": "$membership.plan",
                            "members": {"$sum": 1},
                            "active": {"$sum": {"$cond": [{"$eq": [status_expr, "active"]}, 1, 0]}}}
 >>  },
                {"$project": {"_id": 0, "plan": "$_id", "members": 1, "active": 1,
                              "expired": {"$subtract": ["$members", "$active"]}}},
                {"$sort": {"members": -1}},
            ],
            "expiring_count": [{"$match": expiring}, {"$count": "members"}],
            "expiring_soon": [
                {"$match": expiring},
                {"$sort": {"membership.end_date": 1}},
                {"$limit": args["limit"]},
                {"$project": {"name": 1, "email": 1, "home_branch_id": 1,
                              "plan": "$membership.plan", "end_date": "$membership.end_date",
                              "days_left": {"$dateDiff": {"startDate": now,
                                                          "endDate": "$membership.end_date",
                                                          "unit": "day"}}}},
            ],
        }},
    ]
    result = list(members.aggregate(pipeline))[0]
    return ok({
        "total_members": result["total"][0]["members"] if result["total"] else 0,
        "by_status": result["by_status"],
        "by_plan": result["by_plan"],
        "expiring_soon": {
            "within_days": args["days"],
            "count": result["expiring_count"][0]["members"] if result["expiring_count"] else 0,
            "members": result["expiring_soon"],         # soonest first, up to `limit`
        },
```

```
314     })
```

Raw_Dataset/seed.py

```
"""
seed.py - builds the MongoDB database for the Gym and Fitness Class Booking System.

Sources (Kaggle):
  megaGymDataset.csv                  -> exercises collection
  gym_members_exercise_tracking.csv   -> members collection (one row = one member)

Generated (documented in the Design and Development document):
  branches, trainers (users with role "trainer"), classes with embedded bookings,
  member names/emails/memberships, workout history, exercise reviews.

Usage:
  python seed.py                # insert straight into MongoDB (needs pymongo + bcrypt)
  python seed.py --json-only    # only write mongoimport-ready JSON files to ./seed_output

Every random choice uses a fixed seed, so the script always produces the same data.
"""

import argparse
import csv
import json
import os
import random
import secrets
from datetime import datetime, timedelta, timezone

random.seed(661)

EXERCISES_CSV = "megaGymDataset.csv"
MEMBERS_CSV = "gym_members_exercise_tracking.csv"
DB_NAME = "gymDB"
ENV_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", ".env")


def load_env(path=ENV_FILE):
    """Reads KEY=value lines from the project .env so the connection string stays out of git."""
    if os.path.exists(path):
        with open(path, encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith("#") and "=" in line:
                    key, value = line.split("=", 1)
                    os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))


load_env()
MONGO_URI = os.environ.get("MONGODB_URI")
DEFAULT_PASSWORD = "Password123!"      # every seeded account uses this password
TODAY = datetime.now(timezone.utc).replace(hour=0, minute=0, second=0, microsecond=0)  # schedule is
 >>    built around the day you run it
HISTORY_WEEKS = 8                      # weeks of past workouts and classes
FUTURE_WEEKS = 4                       # weeks of upcoming classes


# ---------------------------------------------------------------- helpers

class Oid:
    """Stand-in for a MongoDB ObjectId so the build step needs no extra libraries."""
    def __init__(self):
        self.hex = format(int(datetime.now(timezone.utc).timestamp()), "08x") + secrets.token_hex(8)


def new_id():
    return Oid()


def rnd_dt(start, end):
    return start + timedelta(seconds=random.randint(0, int((end - start).total_seconds())))


FIRST_M = ["James", "Jack", "Conor", "Ryan", "Daniel", "Adam", "Ethan", "Liam", "Noah", "Oliver",
           "Matthew", "Aaron", "Sean", "Callum", "Harry", "Thomas", "Ali", "Omar", "Hassan", "Arjun"
 >>   ,
           "Lucas", "Mark", "Paul", "Niall", "Ciaran", "Patrick", "Owen", "Rory", "Samuel", "David"]
FIRST_F = ["Emma", "Sophie", "Grace", "Aoife", "Chloe", "Ella", "Lucy", "Olivia", "Amelia", "Erin",
           "Hannah", "Niamh", "Sarah", "Katie", "Ava", "Mia", "Zara", "Fatima", "Aisha", "Priya",
           "Rachel", "Laura", "Megan", "Ciara", "Orla", "Isla", "Ruby", "Holly", "Anna", "Leah"]
LAST = ["Murphy", "Kelly", "O'Neill", "Smith", "Wilson", "Campbell", "Stewart", "Thompson", "Doherty
```

```
 >>   ",
 77          "Quinn", "McLaughlin", "Robinson", "Johnston", "Moore", "Brown", "Martin", "Walsh", "Byrne",
 78          "Khan", "Ahmed", "Patel", "Hughes", "Gallagher", "Kennedy", "Reid", "Graham", "Bell", "Scott
 >>   ",
 79          "Hamilton", "Boyle", "Fitzpatrick", "McCann", "Burns", "Ferguson", "Watson", "Clarke"]
 80
 81  used_emails = set()
 82
 83
 84  def make_person(gender=None):
 85      gender = gender or random.choice(["Male", "Female"])
 86      first = random.choice(FIRST_M if gender == "Male" else FIRST_F)
 87      last = random.choice(LAST)
 88      base = f"{first}.{last}".lower().replace("'", "")
 89      email, n = f"{base}@example.com", 1
 90      while email in used_emails:
 91          n += 1
 92          email = f"{base}{n}@example.com"
 93      used_emails.add(email)
 94      return f"{first} {last}", email, email.split("@")[0]
 95
 96
 97  # ---------------------------------------------------------------- branches
 98
 99  # Real coordinates (approximate centre of each area). GeoJSON uses [longitude, latitude].
100  BRANCHES = [
101      ("Belfast City Centre", "Belfast", "BT1", -5.9301, 54.5973),
102      ("Titanic Quarter", "Belfast", "BT3", -5.9093, 54.6082),
103      ("Queen's Quarter", "Belfast", "BT7", -5.9342, 54.5844),
104      ("Lisburn", "Lisburn", "BT28", -6.0580, 54.5162),
105      ("Bangor", "Bangor", "BT20", -5.6680, 54.6600),
106      ("Derry~Londonderry", "Derry~Londonderry", "BT48", -7.3086, 54.9966),
107  ]
108  FACILITIES = ["Free weights", "Cardio zone", "Spin studio", "Yoga studio", "Sauna", "Swimming pool",
109                "Lockers", "Parking", "Cafe", "Functional training rig", "Showers"]
110
111
112  def build_branches():
113      out = []
114      for name, city, postcode, lng, lat in BRANCHES:
115          out.append({
116              "_id": new_id(),
117              "name": f"PulseFit {name}",
118              "address": {"area": name, "city": city, "postcode": postcode},
119              "location": {"type": "Point", "coordinates": [lng, lat]},
120              "opening_hours": {"weekdays": "06:00-22:00", "weekends": "08:00-20:00"},
121              "facilities": sorted(random.sample(FACILITIES, random.randint(5, 9))),
122              "capacity": random.choice([120, 150, 180, 200]),
123              "phone": f"028 90{random.randint(100000, 999999)}",
124              "created_at": TODAY - timedelta(days=random.randint(400, 1500)),
125          })
126      return out
127
128
129  # ---------------------------------------------------------------- exercises
130
131  def clean_rating(value):
132      """The source uses blank and 0.0 for 'not rated'; both become null."""
133      try:
134          r = float(value)
135      except (TypeError, ValueError):
136          return None
137      return r if r > 0 else None
138
139
140  def build_exercises():
141      out, seen = [], set()
142      with open(EXERCISES_CSV, newline="", encoding="utf-8") as f:
143          for row in csv.DictReader(f):
144              title = row["Title"].strip()
145              if title.lower() in seen:          # 9 duplicate titles in the source
146                  continue
147              seen.add(title.lower())
148              desc = row["Desc"].strip() or None
149              out.append({
150                  "_id": new_id(),
151                  "title": title,
152                  "description": desc,
153                  "type": row["Type"].strip(),
154                  "body_part": row["BodyPart"].strip(),
```

```
                "equipment": row["Equipment"].strip() or "Other",
                "level": row["Level"].strip(),
                "rating": clean_rating(row["Rating"]),
                "reviews": [],
            })
    return out


# ---------------------------------------------------------------- trainers + admin accounts

CLASS_TYPES = {
    # workout type from the members dataset -> class titles, duration (min), level
    "Yoga": [("Vinyasa Flow", 60), ("Yin Yoga", 60), ("Power Yoga", 45), ("Pilates Core", 45)],
    "HIIT": [("HIIT Blast", 30), ("Tabata", 30), ("Bootcamp", 45), ("Circuit Training", 45)],
    "Cardio": [("Spin", 45), ("Rhythm Ride", 45), ("Cardio Kickboxing", 45), ("Step Aerobics", 45)],
    "Strength": [("Body Pump", 60), ("Kettlebell Strength", 45), ("Powerlifting Basics", 60),
                 ("Functional Strength", 45)],
}


def build_staff(branches):
    users, trainers = [], []
    name, email, username = "Site Admin", "admin@pulsefit.example.com", "admin"
    users.append({"_id": new_id(), "username": username, "email": email, "name": name,
                  "role": "admin", "created_at": TODAY - timedelta(days=500)})
    for b in branches:
        for _ in range(3):
            name, email, username = make_person()
            specialities = random.sample(list(CLASS_TYPES), 2)
            t = {"_id": new_id(), "username": username, "email": email, "name": name,
                 "role": "trainer", "branch_id": b["_id"], "specialities": specialities,
                 "created_at": TODAY - timedelta(days=random.randint(60, 900))}
            users.append(t)
            trainers.append(t)
    return users, trainers


# ---------------------------------------------------------------- members + workouts

PLANS = [("Monthly", 30, 0.55), ("Quarterly", 90, 0.25), ("Annual", 365, 0.15), ("Student", 120, 0.0
 >>  5)]
LEVEL = {"1": "Beginner", "2": "Intermediate", "3": "Expert"}


def pick_plan():
    r, acc = random.random(), 0
    for name, days, p in PLANS:
        acc += p
        if r <= acc:
            return name, days
    return PLANS[0][:2]


def jitter(value, pct, lo=None, hi=None, nd=1):
    v = value * random.uniform(1 - pct, 1 + pct)
    if lo is not None:
        v = max(lo, v)
    if hi is not None:
        v = min(hi, v)
    return round(v, nd)


def build_workouts(row, exercises_by_body):
    """Generate HISTORY_WEEKS of workouts around this member's own averages."""
    freq = int(row["Workout_Frequency (days/week)"])
    dur = float(row["Session_Duration (hours)"]) * 60
    cal = float(row["Calories_Burned"])
    avg_bpm = int(row["Avg_BPM"])
    max_bpm = int(row["Max_BPM"])
    main_type = row["Workout_Type"]
    workouts = []
    start = TODAY - timedelta(weeks=HISTORY_WEEKS)
    for week in range(HISTORY_WEEKS):
        days = sorted(random.sample(range(7), min(7, max(1, freq + random.choice([-1, 0, 0, 0, 1]))))
 >>  ))
        for d in days:
            wtype = main_type if random.random() < 0.7 else random.choice(list(CLASS_TYPES))
            minutes = jitter(dur, 0.15, 20, 150, 0)
            w = {
                "_id": new_id(),
```

```
                "date": start + timedelta(weeks=week, days=d, hours=random.randint(6, 20)),
                "workout_type": wtype,
                "duration_min": int(minutes),
                "calories_burned": int(cal * minutes / dur * random.uniform(0.9, 1.1)),
                "avg_bpm": int(jitter(avg_bpm, 0.05, 90, max_bpm, 0)),
                "max_bpm": int(jitter(max_bpm, 0.03, avg_bpm + 5, 205, 0)),
                "exercises": [],
            }
            if wtype in ("Strength", "HIIT"):
                for body in random.sample(list(exercises_by_body), random.randint(3, 5)):
                    ex = random.choice(exercises_by_body[body])
                    w["exercises"].append({
                        "exercise_id": ex["_id"], "title": ex["title"],
                        "sets": random.randint(2, 5), "reps": random.choice([6, 8, 10, 12, 15]),
                        "weight_kg": None if ex["equipment"] == "Body Only"
                        else random.choice([5, 7.5, 10, 12.5, 15, 20, 25, 30, 40, 50, 60, 80]),
                    })
            workouts.append(w)
    return workouts


def build_members(branches, exercises):
    strength = [e for e in exercises if e["type"] in ("Strength", "Powerlifting", "Plyometrics")]
    exercises_by_body = {}
    for e in strength:
        exercises_by_body.setdefault(e["body_part"], []).append(e)
    exercises_by_body = {k: v for k, v in exercises_by_body.items() if len(v) >= 5}

    members, users = [], []
    with open(MEMBERS_CSV, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            name, email, username = make_person(row["Gender"])
            plan, days = pick_plan()
            if random.random() < 0.8:      # 80% active: current period covers today
                end = TODAY + timedelta(days=random.randint(1, days))
            else:                          # 20% lapsed in the last two months
                end = TODAY - timedelta(days=random.randint(1, 60))
            start = end - timedelta(days=days)
            joined = min(start, TODAY - timedelta(days=random.randint(60, 900)))
            m = {
                "_id": new_id(),
                "name": name,
                "email": email,
                "gender": row["Gender"],
                "age": int(row["Age"]),
                "home_branch_id": random.choice(branches)["_id"],
                "body_metrics": {
                    "weight_kg": float(row["Weight (kg)"]),
                    "height_m": float(row["Height (m)"]),
                    "bmi": float(row["BMI"]),
                    "fat_percentage": float(row["Fat_Percentage"]),
                    "resting_bpm": int(row["Resting_BPM"]),
                },
                "preferred_workout": row["Workout_Type"],
                "experience_level": LEVEL[row["Experience_Level"]],
                "workout_frequency": int(row["Workout_Frequency (days/week)"]),
                "water_intake_l": float(row["Water_Intake (liters)"]),
                "membership": {
                    "plan": plan, "start_date": start, "end_date": end,
                    "status": "active" if end >= TODAY else "expired",
                },
                "workouts": [w for w in build_workouts(row, exercises_by_body) if w["date"] <= end],
                "joined_at": joined,
            }
            members.append(m)
            users.append({"_id": new_id(), "username": username, "email": email, "name": name,
                          "role": "member", "member_id": m["_id"], "created_at": start})
    return members, users


# ---------------------------------------------------------------- classes + bookings

SLOTS = [(0, 7), (0, 18), (1, 12), (1, 19), (2, 7), (2, 18), (3, 18), (4, 7), (4, 17), (5, 10), (6,
 >>   11)]


def build_classes(branches, trainers, members):
    by_branch = {}
    for m in members:
        by_branch.setdefault(m["home_branch_id"].hex, []).append(m)
```

```
    trainers_by_branch = {}
    for t in trainers:
        trainers_by_branch.setdefault(t["branch_id"].hex, []).append(t)

    classes = []
    first_monday = TODAY - timedelta(days=TODAY.weekday(), weeks=HISTORY_WEEKS)
    for b in branches:
        local_members = by_branch.get(b["_id"].hex, [])
        timetable = []                                  # same weekly timetable every week
        for day, hour in SLOTS:
            trainer = random.choice(trainers_by_branch[b["_id"].hex])
            ctype = random.choice(trainer["specialities"])
            title, minutes = random.choice(CLASS_TYPES[ctype])
            timetable.append((day, hour, trainer, ctype, title, minutes,
                              random.choice([12, 15, 20, 25]),
                              random.choice(["Beginner", "All levels", "Intermediate", "Advanced"]))
  >>   )
        for week in range(HISTORY_WEEKS + FUTURE_WEEKS):
            for day, hour, trainer, ctype, title, minutes, capacity, level in timetable:
                start = first_monday + timedelta(weeks=week, days=day, hours=hour)
                past = start < TODAY
                # members who like this workout type are more likely to book
                pool = [m for m in local_members if m["preferred_workout"] == ctype] * 3 + local_mem
  >>   bers
                demand = random.uniform(0.45, 1.05) if past else random.uniform(0.1, 0.8)
                wanted = min(capacity, int(capacity * demand))
                chosen, seen = [], set()
                random.shuffle(pool)
                for m in pool:
                    if len(chosen) >= wanted:
                        break
                    if m["membership"]["end_date"] < start:      # lapsed members can't book
                        continue
                    if m["_id"].hex not in seen:
                        seen.add(m["_id"].hex)
                        chosen.append(m)
                bookings = []
                for m in chosen:
                    cancelled = random.random() < 0.08
                    bookings.append({
                        "_id": new_id(),
                        "member_id": m["_id"],
                        "member_name": m["name"],
                        "booked_at": start - timedelta(days=random.randint(0, 10), hours=random.rand
  >>   int(1, 20)),
                        "status": "cancelled" if cancelled else "booked",
                        "attended": (None if not past or cancelled else random.random() < 0.85),
                    })
                active = sum(1 for bk in bookings if bk["status"] == "booked")
                classes.append({
                    "_id": new_id(),
                    "title": title,
                    "category": ctype,
                    "description": f"{title} is a {minutes}-minute {ctype.lower()} class for {level.
  >>   lower()} members.",
                    "branch_id": b["_id"],
                    "branch_name": b["name"],
                    "trainer_id": trainer["_id"],
                    "trainer_name": trainer["name"],
                    "start_time": start,
                    "duration_min": minutes,
                    "level": level,
                    "capacity": capacity,
                    "booked_count": active,
                    "status": "completed" if past else "scheduled",
                    "bookings": bookings,
                })
    return classes


# ---------------------------------------------------------------- reviews

REVIEW_TEXT = {
    5: ["Brilliant exercise, really feel it working.", "One of my favourites, great results.",
        "Easy to follow and very effective."],
    4: ["Good exercise, took a few tries to get the form right.", "Solid movement, would recommend."
  >>   ,
        "Works well as part of a circuit."],
    3: ["Decent but nothing special.", "Okay, a bit awkward with the equipment.",
        "Fine, but I prefer other variations."],
```

```
    2: ["Hard to get the form right.", "Not very comfortable for me."],
    1: ["Hurt my wrists, would not do again."],
}


def add_reviews(exercises, members):
    for ex in random.sample(exercises, int(len(exercises) * 0.35)):
        for m in random.sample(members, random.randint(1, 4)):
            stars = random.choices([5, 4, 3, 2, 1], weights=[35, 35, 18, 8, 4])[0]
            ex["reviews"].append({
                "_id": new_id(),
                "member_id": m["_id"],
                "member_name": m["name"],
                "stars": stars,
                "comment": random.choice(REVIEW_TEXT[stars]),
                "created_at": rnd_dt(TODAY - timedelta(days=300), TODAY),
            })


# ---------------------------------------------------------------- output

def to_extended_json(obj):
    """Converts Oid/datetime into MongoDB Extended JSON for mongoimport."""
    if isinstance(obj, Oid):
        return {"$oid": obj.hex}
    if isinstance(obj, datetime):
        return {"$date": obj.strftime("%Y-%m-%dT%H:%M:%SZ")}
    if isinstance(obj, dict):
        return {k: to_extended_json(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [to_extended_json(v) for v in obj]
    return obj


def to_bson(obj):
    from bson import ObjectId
    if isinstance(obj, Oid):
        return ObjectId(obj.hex)
    if isinstance(obj, dict):
        return {k: to_bson(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [to_bson(v) for v in obj]
    return obj


def write_json(collections, folder="seed_output"):
    try:
        import bcrypt
        hashed = bcrypt.hashpw(DEFAULT_PASSWORD.encode(), bcrypt.gensalt()).decode()
        for u in collections["users"]:
            u["password"] = hashed
    except ImportError:
        print("  bcrypt not installed: users.json has no passwords (pip install bcrypt to add them)"
  >>   )
    os.makedirs(folder, exist_ok=True)
    for name, docs in collections.items():
        path = os.path.join(folder, f"{name}.json")
        with open(path, "w", encoding="utf-8") as f:
            json.dump([to_extended_json(d) for d in docs], f, ensure_ascii=False)
        print(f"  wrote {path} ({len(docs)} documents)")


def insert_mongo(collections):
    import bcrypt
    from pymongo import MongoClient, ASCENDING, TEXT, GEOSPHERE

    if not MONGO_URI:
        raise SystemExit("MONGODB_URI is not set: add it to the .env file in the project root")

    # One hash for all seeded accounts keeps seeding fast; real sign-ups hash individually.
    hashed = bcrypt.hashpw(DEFAULT_PASSWORD.encode(), bcrypt.gensalt())
    for u in collections["users"]:
        u["password"] = hashed

    db = MongoClient(MONGO_URI)[DB_NAME]
    for name, docs in collections.items():
        db[name].drop()
        db[name].insert_many([to_bson(d) for d in docs])
        print(f"  {name}: {len(docs)} documents")
    db.blacklist.drop()
```

```
466
467      db.branches.create_index([("location", GEOSPHERE)])
468      db.exercises.create_index([("title", TEXT), ("description", TEXT)])
469      db.exercises.create_index([("body_part", ASCENDING), ("equipment", ASCENDING), ("level", ASCENDI
  >>   NG)])
470      db.members.create_index("email", unique=True)
471      db.users.create_index("username", unique=True)
472      db.users.create_index("email", unique=True)
473      db.classes.create_index([("branch_id", ASCENDING), ("start_time", ASCENDING)])
474      db.classes.create_index("bookings.member_id")
475      db.blacklist.create_index("token", unique=True)
476      print("  indexes created")
477
478
479  def main():
480      parser = argparse.ArgumentParser()
481      parser.add_argument("--json-only", action="store_true",
482                          help="write JSON files instead of inserting into MongoDB")
483      args = parser.parse_args()
484
485      print("Building data...")
486      branches = build_branches()
487      exercises = build_exercises()
488      staff, trainers = build_staff(branches)
489      members, member_users = build_members(branches, exercises)
490      classes = build_classes(branches, trainers, members)
491      add_reviews(exercises, members)
492
493      collections = {
494          "branches": branches,
495          "exercises": exercises,
496          "members": members,
497          "classes": classes,
498          "users": staff + member_users,
499      }
500      if args.json_only:
501          write_json(collections)
502      else:
503          insert_mongo(collections)
504      print(f"Done. Every seeded account's password is: {DEFAULT_PASSWORD}")
505
506
507  if __name__ == "__main__":
508      main()
```